

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN MÔN HỌC
ĐỀ TÀI: THIẾT KẾ BỘ TĂNG TỐC XỬ LÝ NHÂN MA
TRẦN THUẦN TÚY TRONG MÔ HÌNH QWEN2 ĐÃ
LƯỢNG TỬ HÓA

Môn học: CE213.Q23 – Thiết kế hệ thống số với HDL

Giảng viên hướng dẫn: Hồ Ngọc Diễm

Thực hiện bởi nhóm 29, bao gồm:

- | | | |
|---------------------------|----------|-------------|
| 1. Nguyễn Đặng Phương Duy | 23520372 | Trưởng nhóm |
| 2. Vũ Đại Dương | 23520359 | Thành viên |

Thời gian thực hiện: 14/03/2026 – 22/05/2026

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án môn học này, em đã nhận được nhiều sự hướng dẫn và hỗ trợ quý báu từ quý thầy cô và bạn bè.

Em xin gửi lời cảm ơn chân thành đến thầy Phạm Hoài Luân và cô Hồ Ngọc Diễm đã tận tình hướng dẫn, góp ý và hỗ trợ em trong suốt quá trình thực hiện đề tài. Những kiến thức và kinh nghiệm mà thầy cô chia sẻ đã giúp em hoàn thiện đồ án một cách tốt hơn.

Em cũng xin cảm ơn quý thầy cô trong khoa đã truyền đạt những kiến thức nền tảng trong suốt quá trình học tập, tạo điều kiện để em có thể thực hiện đề tài này.

Cuối cùng, em xin cảm ơn gia đình và bạn bè đã luôn động viên và hỗ trợ em trong quá trình học tập cũng như hoàn thành đồ án.

Mặc dù đã cố gắng hoàn thiện trong khả năng cho phép, đồ án khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp từ quý thầy cô để có thể tiếp tục hoàn thiện hơn trong tương lai.

MỤC LỤC

LỜI CẢM ƠN.....	2
MỤC LỤC	3
DANH SÁCH CHỮ VIẾT TẮT.....	5
DANH SÁCH HÌNH.....	7
DANH SÁCH BẢNG.....	8
PHÂN CÔNG CÔNG VIỆC	9
LỜI MỞ ĐẦU	10
Chương I. TỔNG QUAN.	11
1. Giới thiệu đề tài.	11
1.1. Bối cảnh và nội dung chính.	11
1.2. Lý do chọn đề tài.	12
1.3. Các môn học liên quan.	13
2. Cơ sở lý thuyết.....	14
2.1. Triển khai LLM trên KIT AMD Kria KV260.	14
2.2. Mô hình ngôn ngữ Qwen2 và vai trò của phép nhân ma trận.	15
2.3. Lượng tử hóa đối xứng.	15
2.4. Thuật toán nhân ma trận phân khối.	17
2.5. Kiến trúc mảng tâm thu và luồng dữ liệu Output Stationary.....	17
2.6. Kỹ thuật đệm Ping-Pong trên BRAM.	19
2.7. Lỗi điều phối dữ liệu phần cứng AXI-DMA.	20
2.8. Giao thức giao tiếp Bus họ AMBA AXI4.....	21
2.9. Tái chuẩn hóa và đánh giá	23
Chương II. KIẾN TRÚC HỆ THỐNG	24
Chương III. THIẾT KẾ HỆ THỐNG PHẦN CỨNG.....	27
1. Thiết kế bộ tăng tốc nhân ma trận.	27
1.1. Kiến trúc Top-level của bộ tăng tốc.....	27
1.2. Đệm dữ liệu đầu vào.....	27
1.3. Điều khiển luồng	28
1.4. Mạng làm lệch dữ liệu	29

1.5. Lỗi tính toán.....	30
1.6. Xử lý hậu kỳ.	30
2. Triển khai hệ thống.	34
3. Kiểm tra và phân tích.....	35
3.1. Kiểm tra chức năng.....	35
3.2. Kiểm tra timing.....	40
3.3. Phân tích PPA của hệ thống.	41
Chương IV. THỰC NGHIỆM HỆ THỐNG TRÊN FPGA.	45
1. Trích xuất ma trận đầu vào và đánh giá dữ liệu lượng tử hóa.	45
2. Tích hợp phần mềm điều khiển.	46
2.1. Ánh xạ bộ nhớ vật lý không gian người dùng (Memory Mapping)	46
2.2. Khởi tạo dữ liệu và cấu hình tham số động.....	46
2.3. Đóng gói dữ liệu theo kiến trúc Systolic Array.	47
2.4. Điều khiển truyền nhận DMA và cơ chế Polling an toàn.....	47
2.5. Mô hình đối chiếu phần mềm.	47
2.6. Đánh giá hiệu năng vi mô.....	47
3. Vận hành bộ tăng tốc trên KV260.	48
3.1. So sánh thời gian xử lý hệ thống trên testbench và trên KV260.....	49
3.2. Kiểm tra kết quả các ma trận lớn trên KV260.....	49
Chương V. ĐÁNH GIÁ HỆ THỐNG.	51
1. Đánh giá hiệu năng xử lý giữa PS và bộ tăng tốc PL trên KV260.....	51
2. Đánh giá tính đúng đắn và hiệu năng với Python	52
2.1. Xác thực chéo logic phần cứng và độ chính xác Bit-True	52
2.2. Phân tích sai số và chất lượng lượng tử hóa cấu trúc LLM	53
Chương VI. KẾT LUẬN.	54
PHỤ LỤC	55
NGUỒN THAM KHẢO	56

DANH SÁCH CHỮ VIẾT TẮT

STT	Từ viết tắt	Ý nghĩa
1	AI	Artificial Intelligence
2	AXI	Advanced eXtensible Interface
3	BRAM	Block Random Access Memory
4	CPU	Central Processing Unit
5	DMA	Direct Memory Access
6	DSP	Digital Signal Processing
7	FFN	Feed-Forward Network
8	FIFO	First In First Out
9	FPGA	Field-Programmable Gate Array
10	FSM	Finite State Machine
11	FWFT	First-Word Fall-Through
12	GEMM	General Matrix Multiply
13	GPU	Graphic Processing Unit
14	HDL	Hardware Description Language
15	LLM	Large Language Model
16	MAC	Multiply-Accumulate
17	MatMul	Matrix Multiplication
18	MM	Memory-Mapped
19	OS	Output Stationary
20	PEs	Processing Elements
21	PL	Programmable Logic
22	PS	Processing System

23	SA	Systolic Array
24	SIMD	Single Instruction, Multiple Data
25	STA	Static Timing Analysis

DANH SÁCH HÌNH

Hình 1. Kết quả phân tích tỷ trọng ma trận của các lớp có trong Qwen2	11
Hình 2. KIT AMD Kria KV260.....	14
Hình 3. Tổng quan về thực hiện Tiling trên kiến trúc GEMM [12]	17
Hình 4. Kiến trúc tương quan của Systolic Array	18
Hình 5. Sơ đồ tổng quan của Ping-pong Buffer [17].....	19
Hình 6. Block Diagram của AXI DMA [15]	21
Hình 7. Tổng quan hệ thống	25
Hình 8. Tổng quan kiến trúc của Top-level	27
Hình 9. FSM của module systolic_dataflow_ctrl	28
Hình 10. Minh họa skew network cho ma trận 4x4	29
Hình 11. Thiết kế cụ thể cho module pe_wrapper.....	30
Hình 12. FSM của module output_serializer.....	31
Hình 13. Hệ thống chi tiết được triển khai trong Vivado	34
Hình 14. Cây thư mục của toàn bộ hệ thống	35
Hình 15. Waveform ghi dữ liệu từ AXI DMA vào BRAM buffer	38
Hình 16. Waveform skew dữ liệu nhận từ BRAM	38
Hình 17. Waveform tín hiệu điều khiển SA và lưu kết quả vào FIFO	39
Hình 18. Waveform tái chuẩn hóa dữ liệu và truyền AXI-Stream output.....	39
Hình 19. Phân bố tài nguyên theo từng module trong hệ thống.....	42
Hình 20. Kết quả phân tích hệ thống.....	43
Hình 21. Kiểm tra kích thước ma trận đã trích xuất.....	45
Hình 22. Kết quả đối chiếu chất lượng lượng tử hóa đầu vào.....	45
Hình 23. Thời gian xử lý nhân ma trận 16x512x16 bằng testbench	49
Hình 24. Thời gian phần cứng xử lý 16x512x16 trên KV260	49
Hình 25. Kết quả xử lý 512x512 trên KV260	50
Hình 26. Kết quả xử lý 5120x1280x5120 trên KV260	50
Hình 27. Kết quả kiểm tra Bit-True.....	52
Hình 28. Kết quả đánh giá giải lượng tử kết quả GEMM	53

DANH SÁCH BẢNG

Bảng 1. Phân tích FSM systolic_dataflow_ctrl	29
Bảng 2. Phân tích FSM output_serializer	31
Bảng 3. Kết quả hiển thị trên Console.....	37
Bảng 4. Kết quả Static Timing Analysis sau Implementation	40
Bảng 5. Mức sử dụng tài nguyên FPGA sau Implementation	41

PHÂN CÔNG CÔNG VIỆC

Thành viên	Công việc	Đóng góp (%)
Nguyễn Đăng Phương Duy	- Phân tích model Qwen2 - Thiết kế ping-pong buffer - Thiết kế test bench - Thiết kế code C - Nạp kiểm thử trên FPGA - Thiết kế slide báo cáo	50%
Vũ Đại Dương	- Thiết kế các module còn lại - Phân tích Waveform - Thực hiện phân tích bằng code Python - Viết báo cáo	50%

LỜI MỞ ĐẦU

Trong các mô hình trí tuệ nhân tạo hiện đại, đặc biệt là Large Language Model (LLM), phép nhân ma trận (GEMM) chiếm phần lớn khối lượng tính toán và băng thông bộ nhớ. Khi kích thước mô hình ngày càng tăng, việc xử lý bằng CPU hoặc GPU truyền thống trên các thiết bị biên gặp nhiều hạn chế về hiệu năng, điện năng tiêu thụ và chi phí triển khai. Vì vậy, FPGA đang trở thành một hướng tiếp cận tiềm năng nhờ khả năng xử lý song song, tái cấu hình linh hoạt và tối ưu hiệu năng trên mỗi watt.

Xuất phát từ yêu cầu đó, đồ án thực hiện nghiên cứu và xây dựng bộ gia tốc GEMM trên nền tảng AMD Kria KV260 nhằm tăng tốc các phép toán cốt lõi trong AI và LLM. Hệ thống sử dụng kiến trúc Systolic Array 16×16 theo cơ chế Output Stationary kết hợp lượng tử hóa INT8 nhằm giảm dung lượng lưu trữ và tăng thông lượng xử lý. Thiết kế tích hợp các thành phần như Ping-Pong BRAM Buffer, AXI DMA/AXI4-Stream, FIFO FWFT, module cộng dồn và Rescale INT32 về INT8. Phần mềm điều khiển trên ARM Cortex-A53 đảm nhiệm việc thực hiện tiling, cấu hình DMA và điều khiển truyền dữ liệu giữa PS-PL.

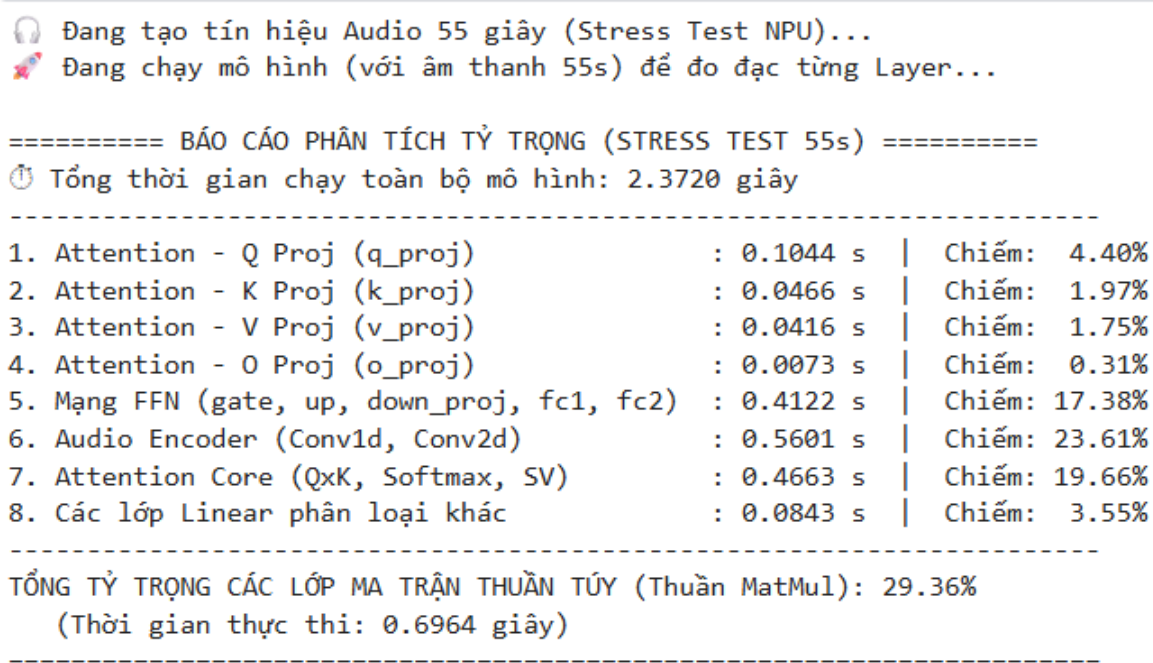
Hệ thống được hiện thực và kiểm thử thực tế trên board KV260. Kết quả thực nghiệm cho thấy bộ gia tốc phần cứng đạt tốc độ xử lý vượt trội so với xử lý thuần CPU, đồng thời hệ thống vận hành bền bỉ và đạt kết quả đầu ra trùng khớp tuyệt đối so với mô hình toán học tham chiếu. Các chỉ số kiểm soát nhiễu lượng tử hóa đều đạt chuẩn yêu cầu công nghiệp, giúp bảo toàn trọn vẹn đặc trưng dữ liệu để triển khai an toàn các mô hình ngôn ngữ lớn mà không gây sụp đổ mô hình.

Chương I. TỔNG QUAN.

1. Giới thiệu đề tài.

1.1. Bối cảnh và nội dung chính.

Sự phát triển mạnh mẽ của các mô hình ngôn ngữ lớn (LLM) và mô hình đa phương thức quy mô lớn, tiêu biểu là họ mô hình Qwen2 với dải tham số trải dài từ 0.5 đến 72 tỷ tham số, đã mang lại những bước đột phá vượt bậc trong xử lý ngôn ngữ tự nhiên và tín hiệu âm thanh phức tạp [1]. Kiến trúc cốt lõi cấu thành nên sự thành công của các hệ thống này là mạng Transformer, một mô hình xử lý song song dựa hoàn toàn trên cơ chế Self-Attention. Tuy nhiên, để đòi lấy năng lực biểu diễn ngữ cảnh mạnh mẽ, Transformer đòi hỏi một khối lượng tài nguyên tính toán khổng lồ cùng băng thông bộ nhớ cực lớn, tạo ra thách thức nghiêm trọng khi triển khai trên các thiết bị thực tế [2].



Hình 1. Kết quả phân tích tỷ trọng ma trận của các lớp có trong Qwen2

Để có số liệu thực nghiệm làm cơ sở khoa học, nhóm nghiên cứu đã tiến hành chạy thử nghiệm stress test trực tiếp trên mô hình tinh chỉnh *Qwen2-Audio-Dialogue* được cấu hình chạy ở chế độ chính xác cao nhất FP32 để bắt giữ toàn bộ các giá trị ngoại lai (outliers) [3]. Với đầu vào là một đoạn tín hiệu âm thanh thực tế kéo dài 55 giây nhằm ép hệ thống hoạt động ở ngưỡng tải tối đa, kết quả phân tích tỷ trọng cho thấy các phép toán nhân ma trận mật độ cao (Dense Matrix Multiplication - GEMM)—bao gồm các lớp chiếu tuyến tính Q , K , V , O của bộ lõi Attention và các lớp phân tách $gate_proj$, up_proj , $down_proj$ trong mạng khuếch đại Feed-Forward Network (FFN)—chiếm tỷ trọng áp đảo lên tới hơn 29.36% tổng thời gian thực thi của mô hình [3]. Điều

này chứng minh toán tử nhân ma trận chính là nút thắt cổ chai cục bộ làm suy giảm hiệu năng xử lý của toàn bộ hệ thống [3].

Đề tài này tập trung vào việc nghiên cứu, thiết kế và hiện thực hóa một bộ tăng tốc phần cứng chuyên dụng trên nền tảng vi mạch lập trình được FPGA nhằm giải quyết triệt để nút thắt cổ chai tính toán GEMM cho họ mô hình Qwen2. Quy trình thực hiện nghiên cứu được xây dựng bài bản qua 5 giai đoạn cốt lõi:

1. Trích xuất và đánh giá đặc tính: Thực hiện cơ chế forward-pass trên framework phần mềm để trích xuất các ma trận trọng số (Weights) và kích hoạt (Activations) thực tế, đồng thời áp dụng hệ thống đo lường phần cứng thông qua CUDA Events để định lượng chi phí tính toán của từng phân lớp.
2. Lượng tử hóa tối ưu phần cứng: Nghiên cứu và áp dụng kỹ thuật lượng tử hóa đối xứng tuyệt đối để nén dữ liệu từ định dạng số thực dấu phẩy động 32-bit (FP32) xuống định dạng số nguyên thấp cấp 8-bit (INT8) [4].
3. Thiết kế kiến trúc phần cứng cốt lõi: Thiết kế cấu trúc mảng tâm thu (Systolic Array - SA) kích thước 16×16 hoạt động theo cơ chế luồng dữ liệu cố định ngõ ra (Output Stationary- OS), tích hợp hệ thống đường ống tối ưu hóa xung nhịp. Dữ liệu được điều phối thông qua lõi IP AXI DMA kết hợp bus truyền tải dữ liệu luồng tốc độ cao AXI4-Stream vào hệ thống bộ đệm kép BRAM Ping-Pong.
4. Xử lý hậu kỳ và tái chuẩn hóa phần cứng: Xây dựng module logic tích lũy phân khối phối hợp với bộ nhớ FIFO FWFT (First-Word Fall-Through), cấu hình mạch chuyển đổi định dạng và dịch bit làm tròn để đưa kết quả tích lũy 32-bit (INT32) về lại định dạng 8-bit (INT8) trước khi xuất ngoại vi.
5. Kiểm chứng chéo chuẩn bit: Xây dựng mô hình toán học tương đương (Golden Model) trên môi trường Python để đối chiếu từng bit kết quả thu được từ mạch RTL thực tế trên kit FPGA, đánh giá sai số thông qua các chỉ số công nghiệp như Cosine Similarity, tỷ số tín hiệu trên nhiễu lượng tử (SQNR), RMSE và MAE.

1.2. Lý do chọn đề tài.

Nhóm nghiên cứu quyết định lựa chọn đề tài này xuất phát từ 4 động lực khoa học và thực tiễn cốt lõi sau:

Vượt qua Memory Wall: Theo lý thuyết phân tích hiệu năng của mô hình Roofline Model, các kiến trúc máy tính truyền thống (Von Neumann) khi xử lý các mô hình AI quy mô lớn thường bị giới hạn nghiêm trọng bởi băng thông truy xuất bộ nhớ ngoài trước khi chạm được tới ngưỡng giới hạn năng lực tính toán lý thuyết của chip

[5]. Việc thiết kế một kiến trúc phần cứng chuyên dụng cho phép phân cấp và tái sử dụng dữ liệu triệt để ngay trên chip nhằm phá vỡ giới hạn này.

Tối ưu hóa tài nguyên nhờ lượng tử hóa số nguyên: Các nghiên cứu thực nghiệm quy mô lớn của Google trên bộ xử lý TPU (Tensor Processing Unit) đã chứng minh rằng việc thay thế các đường dẫn tính toán dấu phẩy động bằng mạch toán tử INT8 giúp giảm tới 6 lần năng lượng tiêu thụ và diện tích vi khối silicon, trong khi cho phép tích hợp mật độ bộ nhân cộng (MAC) cao gấp hàng chục lần trên cùng một đơn vị diện tích phần cứng [6].

Ưu thế kiến trúc của Mạng tâm thu: Đối với các toán tử nhân ma trận trù mật, kiến trúc SA chứng minh sự vượt trội hoàn toàn so với kiến trúc SIMD (Single Instruction, Multiple Data) truyền thống trên GPU nhờ cơ chế truyền tải và chia sẻ dữ liệu trực tiếp giữa các phần tử xử lý lân cận (Processing Elements - PEs), giảm thiểu việc truy xuất vào tập thanh ghi trung tâm, từ đó triệt tiêu độ trễ bus và tiết kiệm điện năng [7].

Thúc đẩy xu hướng Edge AI: Việc hiện thực hóa thành công bộ gia tốc GEMM trên các dòng Edge FPGA cấu hình thấp chứng minh tính khả thi của việc đưa các mô hình trí tuệ nhân tạo phức tạp ra vận hành tại các thiết bị đầu cuối với không gian tài nguyên hạn chế và nghiêm ngặt về công suất tiêu thụ [8].

1.3. Các môn học liên quan.

Đề tài là kết quả của sự vận dụng tổng hợp và đào sâu kiến thức nền tảng từ bốn phân nhánh môn học chuyên ngành cốt lõi:

Kiến trúc máy tính & Vi xử lý: Cung cấp cơ sở lý thuyết về tổ chức hệ thống, cấu trúc phân cấp bộ nhớ (Cache, SRAM, DRAM), phân tích xung đột đường ống, cơ chế truy cập bộ nhớ trực tiếp (DMA) và mô hình trao đổi dữ liệu qua hệ thống Bus ngoại vi.

Thiết kế hệ thống số với HDL: Nền tảng kỹ thuật toán học để mô tả cấu trúc phần cứng bằng ngôn ngữ cấu trúc Verilog, thiết kế máy trạng thái mạch logic (FSM), tối ưu hóa mạch tổ hợp và mạch tuần tự, phân tích Timing (Setup/Hold time) và kiểm soát phân bố tài nguyên trên vi mạch FPGA [9].

Thiết kế hệ thống nhúng: Phương pháp luận phối hợp thiết kế đồng thời Phần cứng/Phần mềm, kỹ thuật viết driver điều khiển phần cứng bằng ngôn ngữ C/C++ cấp thấp, cấu hình hệ điều hành Linux nhúng và quản lý bộ nhớ vật lý.

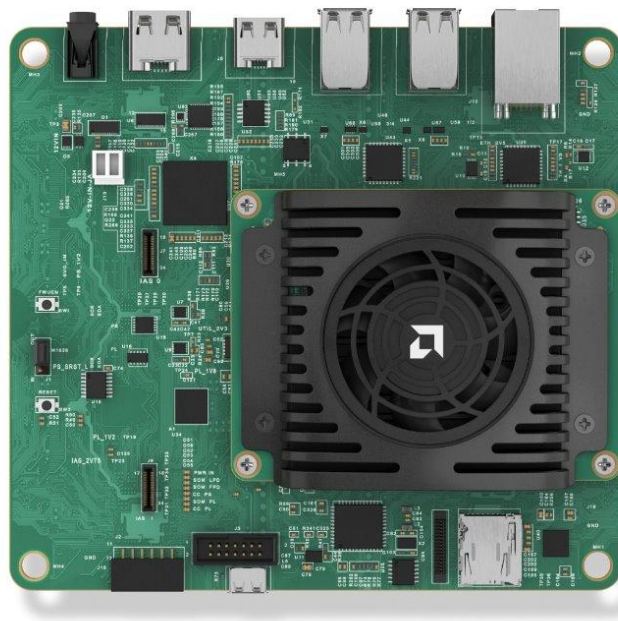
Trí tuệ nhân tạo & Học máy: Cung cấp hiểu biết toán học về cấu trúc mạng nơ-ron, cơ chế Forward Pass, các hàm kích hoạt và nguyên lý tối ưu hóa hàm toán học thông qua các giải pháp thu gọn biểu diễn số liệu (Quantization).

2. Cơ sở lý thuyết.

Để hiện thực hóa hệ thống gia tốc trên, đề tài áp dụng các lý thuyết và công nghệ cốt lõi sau:

2.1. Triển khai LLM trên KIT AMD Kria KV260.

Việc triển khai các nguyên mẫu LLM xuống phần cứng biên đang là xu hướng tất yếu nhằm giảm độ trễ và tăng hiệu năng xử lý tại thiết bị đầu cuối [8]. Trong đồ án này, nhóm lựa chọn AMD Kria KV260 Vision AI Starter Kit làm nền tảng triển khai bộ gia tốc GEMM. KV260 sử dụng SoC Zynq UltraScale+ MPSoC, tích hợp đồng thời hai miền xử lý gồm Processing System (PS) và Programmable Logic (PL) [10], [11]. Kiến trúc này đặc biệt phù hợp với bài toán tăng tốc AI khi cho phép kết hợp xử lý phần mềm linh hoạt với khả năng xử lý song song của phần cứng FPGA.



Hình 2. KIT AMD Kria KV260

Về phía PS, KV260 sử dụng bộ vi xử lý lõi tứ ARM Cortex-A53 64-bit chạy hệ điều hành Linux nhúng Debian, đồng thời tích hợp các ngoại vi như bộ điều khiển DDR4, Ethernet, USB và AXI Interconnect. Board được trang bị 4 GB DDR4 RAM, đủ khả năng lưu trữ các ma trận kích thước lớn phục vụ quá trình kiểm thử. Với dữ liệu INT8, ma trận kích thước 5120×1280 chỉ chiếm khoảng 6.25 MB bộ nhớ, trong khi ma trận đầu ra 5120×5120 chiếm khoảng 25 MB, hoàn toàn nằm trong khả năng lưu trữ của hệ thống DDR4 trên KV260. Trong thiết kế của đề tài, phần PS đảm nhiệm việc sinh dữ liệu đầu vào, thực hiện thuật toán Tiling, đóng gói dữ liệu phù hợp với luồng dữ liệu của Systolic Array, cấu hình AXI DMA và AXI GPIO, đồng thời điều khiển quá trình truyền nhận dữ liệu giữa DDR và PL.

Về phía PL, thiết bị FPGA trên KV260 cung cấp khoảng 256K LUTs, 512K Flip-Flops, 144 Block RAM (BRAM) và 1248 khối DSP48E2 [10]. Đây là các tài nguyên quan trọng phục vụ xây dựng các kiến trúc xử lý song song cho bài toán GEMM. Trong đồ án, phần PL được sử dụng để triển khai Systolic Array 16×16 theo cơ chế Output Stationary, kết hợp với hệ thống Ping-Pong BRAM Buffer, FIFO FWFT, các pipeline MAC, module cộng dồn và khối Rescale $\text{INT32} \rightarrow \text{INT8}$. Quá trình truyền dữ liệu giữa DDR RAM và FPGA được thực hiện thông qua AXI DMA nhằm tối ưu băng thông truyền tải, trong khi AXI GPIO được sử dụng để cấu hình các tham số như K-Dimension, số lượng K-Tile và hệ số dịch Scale Shift. Giao tiếp dữ liệu giữa các module phần cứng sử dụng chuẩn AXI4-Stream nhằm đảm bảo khả năng truyền dữ liệu liên tục với thông lượng cao.

Sự kết hợp giữa PS và PL giúp hệ thống tận dụng được tính linh hoạt của phần mềm và khả năng xử lý song song hiệu năng cao của FPGA, từ đó phù hợp với các bài toán GEMM kích thước lớn trong AI và LLM.

2.2. Mô hình ngôn ngữ Qwen2 và vai trò của phép nhân ma trận.

Mô hình toán học cơ sở của họ mô hình Qwen2 hoạt động dựa trên cơ chế Multi-Head Attention [1]. Tại mỗi lớp Transformer, dữ liệu đầu vào kích hoạt X được nhân với các ma trận trọng số tương ứng W_Q , W_K , W_V nhằm tạo ra ba ma trận đặc trưng: Truy vấn (Query - Q), Khóa (Key - K), và Giá trị (Value - V) theo phương trình [2]:

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V$$

Sau đó, toán tử Self-Attention lõi được thực thi bằng cách tính toán độ tương đồng tích vô hướng giữa các vector đặc trưng, chuẩn hóa thông qua hàm Softmax và nhân tích lũy với ma trận Giá trị V [2]:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

Toàn bộ các pha biến đổi tuyến tính này, kết hợp với các toán tử chiều lớp FFN sau đó ($\text{FFN}(x) = \text{Silu}(xW_{\text{gate}}) \cdot xW_{\text{up}} \cdot W_{\text{down}}$), bản chất đều là chuỗi lặp lại liên tục của các phép nhân ma trận dày đặc (GEMM). Do đó, tăng tốc độ thực thi phần cứng của toán tử MatMul sẽ trực tiếp quyết định thông lượng suy luận (Tokens/giây) và giảm thiểu tối đa độ trễ phản hồi của mô hình Qwen2 [1], [11].

2.3. Lượng tử hóa đối xứng.

Để tối ưu hóa diện tích vi mạch và giảm thiểu tối đa băng thông truyền tải dữ liệu giữa PS và PL, đề tài áp dụng giải pháp lượng tử hóa tuyến tính đối xứng tuyệt đối (Linear Symmetric Quantization) nhằm chuyển đổi dữ liệu từ miền số thực FP32 sang miền số nguyên giới hạn INT8 [4]. Thuật toán ánh xạ một không gian phân bố số

thực liên tục $[-\alpha, \alpha]$ vào dải số nguyên đối xứng $[-127, 127]$ với điểm *Zero-Point* của hệ thực trùng khớp tuyệt đối với giá trị 0 của hệ nguyên [4]:

$$X_{\text{int8}} = \text{clamp}\left(\left\lfloor \frac{X_{\text{fp32}}}{\text{Scale}} \right\rfloor, -128, 127\right)$$

Trong đó, hệ số thu phóng (Scale Factor) được xác định dựa trên giá trị biên độ tuyệt đối cực đại của tensor dữ liệu nhằm bảo toàn tối đa Outliers có ý nghĩa quyết định đến chất lượng suy luận của LLM [4]:

$$\text{Scale} = \frac{\max(|X_{\text{fp32}}|)}{127}$$

Ưu điểm cơ bản của lượng tử hóa đối xứng khi triển khai phần cứng RTL là triệt tiêu hoàn toàn sự xuất hiện của tham số *Zero-Point* ($Z_x = 0$). Trong các cấu trúc lượng tử hóa phi đối xứng (Asymmetric), phép nhân hai phần tử lượng tử hóa đòi hỏi mạch logic phải gánh thêm các bộ cộng đại số phức tạp để bù trừ khoảng lệch toán học:

$$(X_q - Z_x) \times (Y_q - Z_y) = X_q Y_q - X_q Z_y - Y_q Z_x + Z_x Z_y$$

Với lượng tử hóa đối xứng, phương trình toán học tại lõi nhân của cấu trúc PE rút gọn chỉ còn duy nhất một toán tử nhân số nguyên thuần túy ($X_q \times Y_q$). Phép giản lược này giúp tiết kiệm hàng ngàn cổng logic cấu thành bộ cộng, giảm độ trễ đường truyền tới hạn (Critical Path Delay), và cho phép mạch hoạt động ở tần số xung nhịp cao hơn [4], [7].

Sau khi mảng phần cứng thực hiện phép nhân cộng tích lũy, dữ liệu trung gian ngõ ra sẽ mang định dạng số nguyên 32-bit (INT32) để ngăn chặn hiện tượng tràn số. Giá trị này cần được tái chuẩn hóa về lại định dạng INT8 thông qua tích hiệu dụng của hai hệ số scale đầu vào cùng phép dịch bit phần cứng trước khi ghi ngược lại bộ nhớ hệ thống [4]:

$$C_{\text{int8}} = \text{clamp}((C_{\text{int32}} \times \text{scale_effective}) \gg \text{SCALE_SHIFT}, -128, 127)$$

Để đánh giá độ suy hao độ chính xác sau quá trình lượng tử hóa phần cứng, bốn chỉ số đo lường chuẩn công nghiệp được áp dụng:

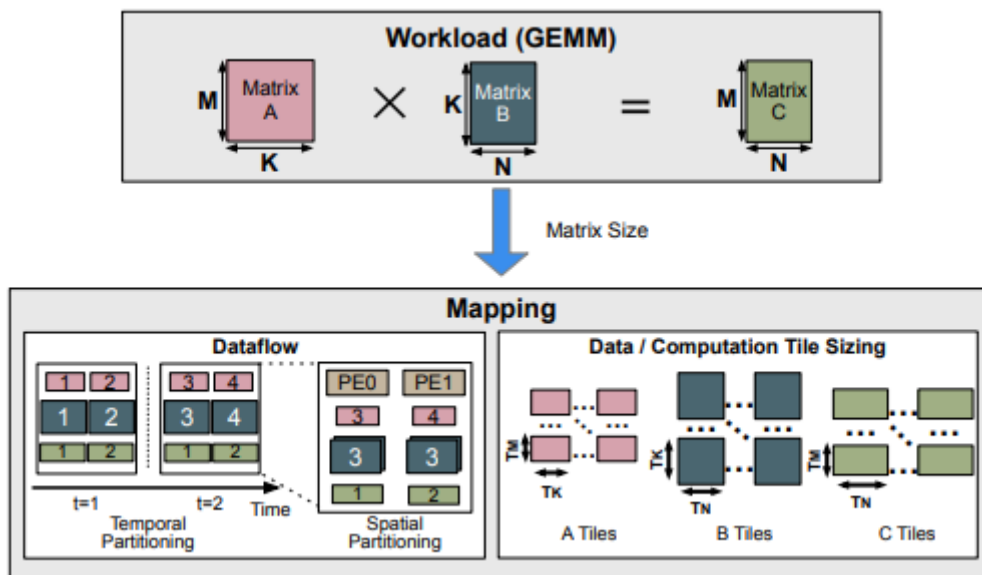
Cosine Similarity: Đo lường độ trùng khớp về mặt hướng của vector kết quả giữa phần cứng và phần mềm.

SQNR (Signal-to-Quantization-Noise Ratio): Tỷ số công suất tín hiệu trên công suất nhiễu lượng tử, định lượng mức độ méo dạng tín hiệu.

MAE (Mean Absolute Error) & RMSE (Root Mean Squared Error): Định lượng biên độ sai lệch tuyệt đối trung bình giữa giá trị thực và giá trị lượng tử hóa.

2.4. Thuật toán nhân ma trận phân khối.

Các kiến trúc Edge FPGA có dung lượng bộ nhớ nội vi on-chip (SRAM/BRAM) rất giới hạn, hoàn toàn không đủ không gian vật lý để lưu trữ đồng thời toàn bộ các ma trận kích thước khổng lồ của mô hình Qwen2 [8], [11]. Do đó, việc áp dụng thuật toán chia khối dữ liệu là bắt buộc [12].



Hình 3. Tổng quan về thực hiện Tiling trên kiến trúc GEMM [12]

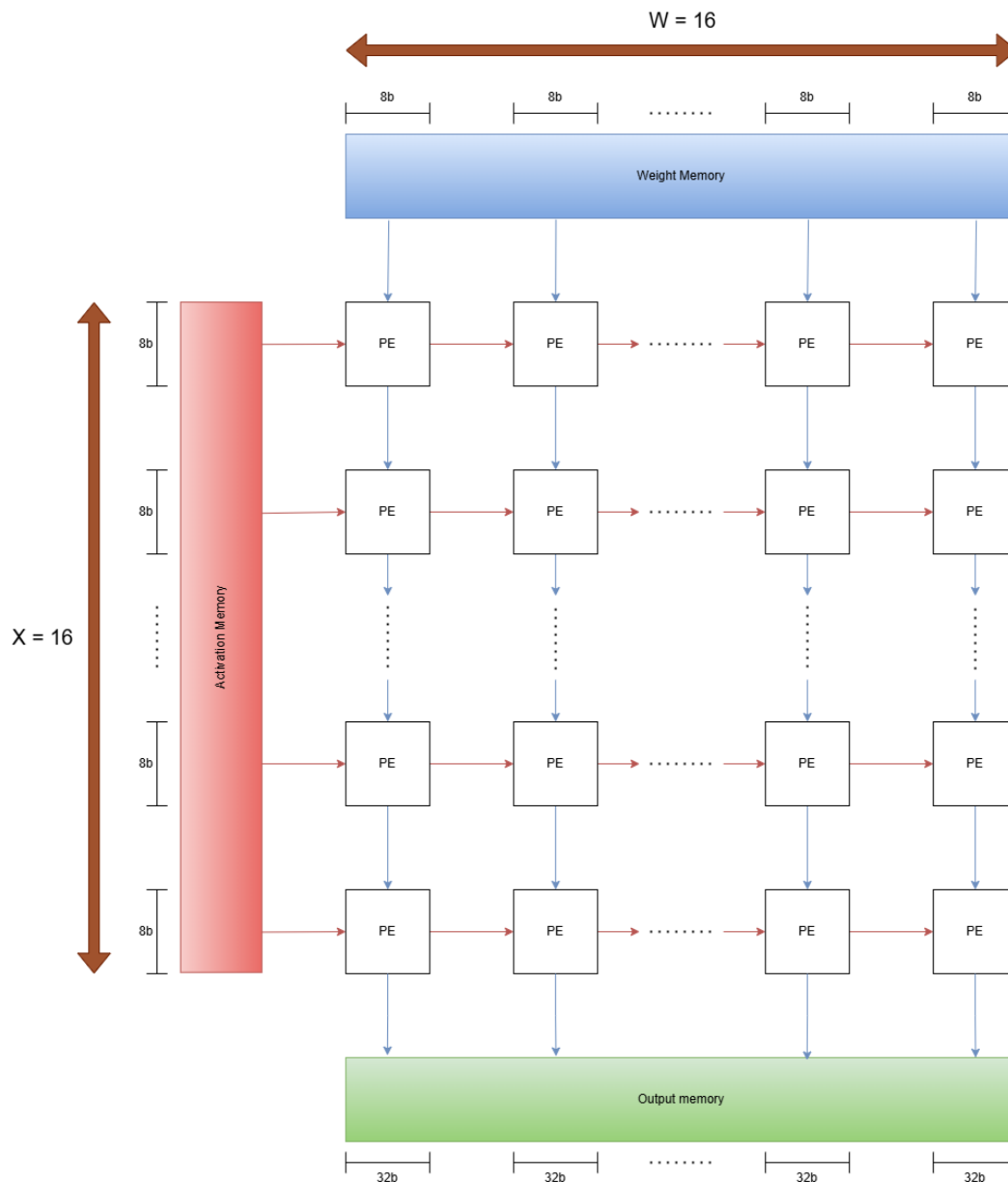
Thuật toán thực hiện phân rã phép nhân ma trận tổng thể $C = A \cdot B$ thành chuỗi các phép nhân ma trận con có kích thước cố định lớp vỏ ngoài khớp với cấu trúc phần cứng là 16×16 [12]. Để tăng cường thông lượng truy cập bộ nhớ, các khối BRAM lưu trữ dữ liệu này trên PL cần được cấu hình dưới dạng Dual-port RAM, cho phép một cổng liên tục nhận dữ liệu nạp từ bus ngoại vi trong khi cổng còn lại cung cấp dữ liệu tức thời cho mảng tính toán [17]. Đồng thời, áp dụng chỉ thị phân mảnh mảng trên thiết kế để tách một khối bộ nhớ logic lớn thành nhiều block vật lý song song, cho phép mảng PE truy xuất đồng thời toàn bộ các phần tử trong cùng một hàng hoặc cột tại mỗi chu kỳ xung nhịp, triệt tiêu hoàn toàn độ trễ khởi tạo dữ liệu [9].

2.5. Kiến trúc mảng tâm thu và luồng dữ liệu Output Stationary.

Kiến trúc mảng tâm thu bao gồm một mạng lưới 2D gồm các phần tử xử lý PE cấu trúc đồng nhất, kết nối cục bộ với nhau theo mô hình lưới [13]. Dữ liệu từ bộ đệm sẽ truyền qua các PE một cách tuần hoàn và đồng bộ theo từng nhịp xung nhịp, cho phép một phần tử dữ liệu sau khi đọc ra từ bộ nhớ được tái sử dụng nhiều lần tại nhiều PE khác nhau, giải quyết triệt để bài toán nghẽn cổ chai băng thông truyền tải dữ liệu của kiến trúc Von Neumann [7], [13].

Đề tài lựa chọn kiến trúc luồng dữ liệu OS cho mảng tâm thu kích thước 16×16 [12]. Trong kiến trúc này, mỗi phần tử PE tại tọa độ (i, j) bên trong mảng sẽ tích hợp

một thanh ghi tích lũy nội vi đảm nhiệm việc lưu giữ cố định giá trị tổng riêng phần của phần tử ma trận kết quả $C[i][j]$ [13]. Tại mỗi chu kỳ xung nhịp, phần tử của ma trận kích hoạt A chảy xuyên qua mảng theo chiều ngang (từ trái sang phải), trong khi phần tử của ma trận trọng số B chảy xuyên qua mảng theo chiều dọc (từ trên xuống dưới) [12], [13]. Các PE thực hiện phép nhân cộng (MAC) giữa hai giá trị nhận được và tích lũy trực tiếp vào thanh ghi tại chỗ [13].



Hình 4. Kiến trúc tương quan của Systolic Array

Ví dụ với phép nhân ma trận 2×2 :

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \times \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} PE0 & PE1 \\ PE2 & PE3 \end{bmatrix}$$

Bảng dưới đây thể hiện sự khác biệt giữa 2 phương pháp

Chu kì	Nhân ma trận thông thường	Systolic Array
1	$PE0 = A \times E$	$PE0 = A \times E$
2	$PE0 = A \times E + B \times G$	$PE0 = A \times E + B \times G$ $PE1 = A \times F$ $PE2 = C \times E$
3	$PE1 = A \times F$	$PE1 = A \times F + B \times H$ $PE2 = C \times E + D \times G$ $PE3 = C \times F$
4	$PE1 = A \times F + B \times H$	$PE3 = C \times F + D \times H$

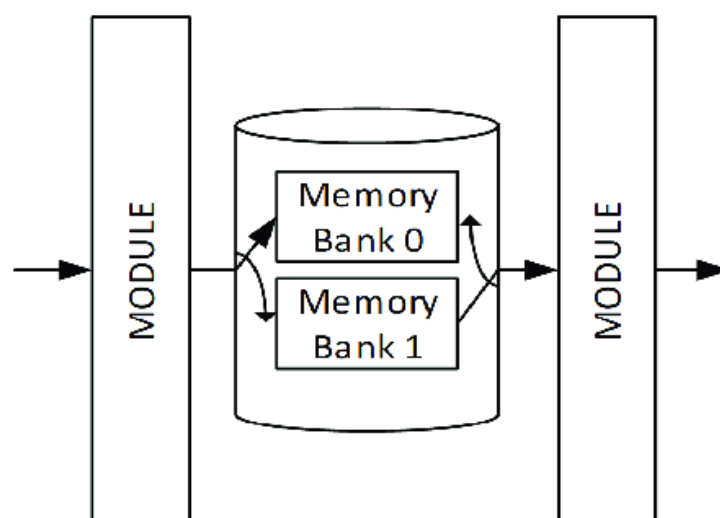
Có thể thấy rằng thuật toán thông thường cần xử lý tuần tự từng phần tử đầu ra, trong khi Systolic Array cho phép nhiều PE thực hiện phép MAC đồng thời. Nhờ đó, SA tận dụng tốt tính song song của phần cứng FPGA, giảm số chu kỳ xử lý và tăng thông lượng hệ thống.

So với kiến trúc SIMD truyền thống trên Graphic Processing Unit (GPU) - vốn yêu cầu đọc/ghi liên tục với hệ thống thanh ghi tập trung, Systolic Array chứng minh sự vượt trội khi xử lý Dense MatMul nhờ khả năng chuyển tiếp dữ liệu cục bộ giữa các PEs mà không cần truy xuất bộ nhớ ngoài [7], [11].

So với kiến trúc Weight Stationary (Cố định trọng số) đòi hỏi phải tích hợp thêm mạch điều khiển nạp trước trọng số và quản lý trạng thái phức tạp, kiến trúc OS cho phép ma trận Weight và Activation được đẩy vào mảng đồng bộ tuyến tính thông qua các cấu trúc thanh ghi dịch đơn giản, tối ưu đường truyền tín hiệu tần số cao [12], [13].

2.6. Kỹ thuật đệm Ping-Pong trên BRAM.

Nhằm ngăn chặn hiện tượng mảng tâm thu rơi vào trạng thái ngừng hoạt động chờ dữ liệu do độ trễ truyền tải của bus ngoại vi, kỹ thuật đệm kép Ping-Pong (Double Buffering) được triển khai trên hệ thống tài nguyên BRAM của phần PL [9], [14].



Hình 5. Sơ đồ tổng quan của Ping-pong Buffer [17]

Cơ chế vận hành của bộ đệm Ping-Pong chia bộ nhớ đệm thành hai phân vùng logic độc lập có kích thước bằng nhau: Phân vùng PING và phân vùng PONG [9]. Mạch điều khiển hoạt động dựa trên nguyên lý tương phản chéo chu kỳ:

Tại chu kỳ N : Systolic Array thực hiện kết nối và đọc quyền đọc dữ liệu từ phân vùng PING để thực hiện các phép toán nhân cộng. Đồng thời, lõi điều khiển DMA ngoại vi thực hiện ghi dữ liệu của khối phân khối tiếp theo ($Tile_{N+1}$) từ bộ nhớ DDR RAM ngoài vào phân vùng PONG.

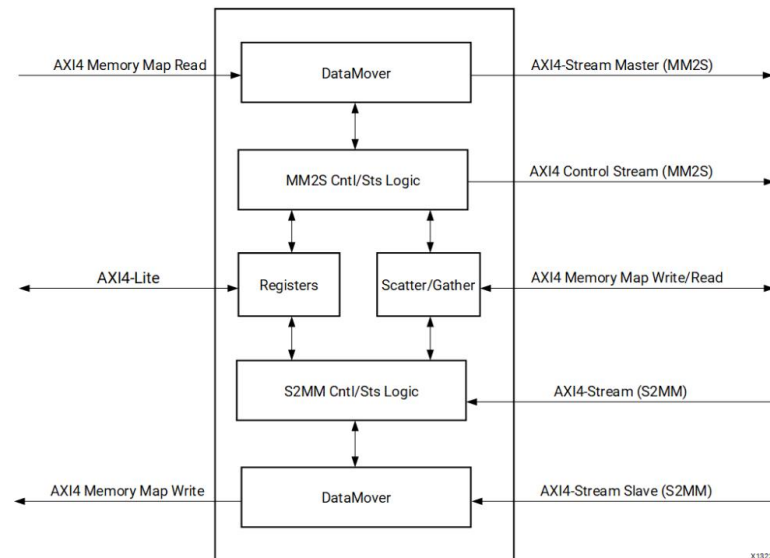
Tại chu kỳ $N+1$: Ngay sau khi mảng tính toán xong và DMA hoàn thành việc nạp dữ liệu, mạch điều khiển FSM thực hiện đảo chéo trạng thái bộ chọn. Lúc này, mảng tâm thu sẽ đọc dữ liệu mới từ phân vùng PONG, trong khi lõi DMA tiếp tục ghi gói đầu dữ liệu của khối kế tiếp vào phân vùng PING.

Giải pháp này cho phép che giấu hoàn toàn độ trễ truyền dữ liệu của hệ thống Bus, đảm bảo mảng tính toán luôn được cung cấp dữ liệu liên tục ở mức thông lượng tối đa [9], [14].

2.7. Lõi điều phối dữ liệu phần cứng AXI-DMA.

Hệ thống điều phối dữ liệu tốc độ cao giữa vùng nhớ DDR4 RAM thuộc PS và các bộ đệm Ping-Pong thuộc PL được hiện thực hóa thông qua lõi IP LogiCORE AXI Direct Memory Access (AXI DMA) [15]. Sự tích hợp của lõi AXI DMA giúp giải phóng hoàn toàn bộ vi xử lý ARM Cortex-A53 khỏi nhiệm vụ sao chép dữ liệu thủ công từng byte theo cách tuần tự, từ đó tối ưu hóa tài nguyên tính toán của CPU cho các tác vụ điều khiển cấp cao khác [15].

Để vận hành tác vụ, phần mềm điều khiển nhúng chỉ cần cấu hình một lần duy nhất vào các thanh ghi chức năng của lõi DMA, bao gồm: địa chỉ con trỏ vùng nhớ nguồn, địa chỉ con trỏ vùng nhớ đích và độ dài của khối dữ liệu cần truyền tải [15]. Sau khi lệnh được kích hoạt, lõi DMA sẽ tự động thiết lập cơ chế chiếm quyền điều khiển bus, tự quản lý việc sinh địa chỉ và thực hiện các chu kỳ truyền dữ liệu hàng loạt tốc độ cao trực tiếp giữa PS và PL thông qua các kênh dịch dịch bộ nhớ chuyên dụng (MM2S - Memory Map to Stream và S2MM - Stream to Memory Map) [15].



Hình 6. Block Diagram của AXI DMA [15]

2.8. Giao thức giao tiếp Bus họ AMBA AXI4

Trong kiến trúc hệ thống SoC hiện đại, họ giao tiếp AMBA AXI4 (Advanced eXtensible Interface 4) do ARM định nghĩa đóng vai trò là xương sống kết nối các thành phần IP phần cứng [16]. Tùy thuộc vào đặc tính băng thông, yêu cầu tài nguyên logic và bản chất của dữ liệu truyền tải, hệ thống tích hợp linh hoạt cả ba biến thể giao thức bus bao gồm: AXI4-Memory Mapped (AXI4-MM), AXI4-Lite và AXI4-Stream [16], [17].

2.8.1. Giao thức AXI4-Memory Mapped (AXI4-MM)

AXI4-Memory Mapped (thường gọi là AXI4 Full) là giao thức bus định hướng bộ nhớ, được thiết kế cho các tác vụ yêu cầu ánh xạ địa chỉ tường minh, hiệu năng cao và băng thông lớn [16]. Kiến trúc của AXI4-MM phân tách hoàn toàn quá trình truyền nhận dữ liệu thành 5 kênh độc lập, mỗi kênh sở hữu một tập hợp các tín hiệu bắt tay độc lập [16]:

Kênh địa chỉ ghi (Write Address Channel)

Kênh dữ liệu ghi (Write Data Channel)

Kênh đáp ứng ghi (Write Response Channel)

Kênh địa chỉ đọc (Read Address Channel)

Kênh dữ liệu đọc (Read Data Channel)

Nhờ cấu trúc song song này, AXI4-MM hỗ trợ khả năng truyền nhận dữ liệu hai chiều toàn phần cùng lúc. Đặc tính cốt lõi của AXI4-MM là hỗ trợ cơ chế truyền hàng loạt: bên Master chỉ cần phát ra địa chỉ cơ sở một lần duy nhất trong chu kỳ thiết lập địa

chỉ ban đầu, sau đó có thể truyền liên tiếp một chuỗi tối đa lên đến 256 Data Words mà không cần lặp lại pha địa chỉ. Giao thức này cũng hỗ trợ các tính năng nâng cao như Out-of-order transactions và xử lý Data interleaving nhằm khai thác tối đa hiệu suất bus [16].

2.8.2. Giao thức AXI4-Lite

AXI4-Lite là một tập hợp con được đơn giản hóa từ giao thức gốc AXI4-MM. Điểm khác biệt lớn nhất của AXI4-Lite là loại bỏ hoàn toàn khả năng truyền dữ liệu hàng loạt. Điều này đồng nghĩa với việc mỗi giao dịch đọc hoặc ghi chỉ cho phép dịch chuyển một từ dữ liệu duy nhất cho mỗi chu kỳ địa chỉ. Giao thức này vẫn duy trì cấu trúc 5 kênh độc lập nhưng lược bỏ phần lớn các tín hiệu điều khiển phụ phức tạp [16].

Do cấu trúc bus tối giản, AXI4-Lite chiếm dụng rất ít tài nguyên phần cứng và đơn giản hóa đáng kể thiết kế mạch logic điều khiển [17]. Trong hệ thống, AXI4-Lite đóng vai trò lý tưởng để PS truy cập, cấu hình và giám sát các thanh ghi chức năng (Control and Status Registers - CSR) hoặc các thiết bị ngoại vi tốc độ thấp có ảnh xạ không gian địa chỉ tĩnh trong PL [16], [17].

2.8.3. Giao thức AXI4-Stream

Khác biệt hoàn toàn với hai giao thức ánh xạ bộ nhớ nêu trên, giao thức AXI4-Stream được thiết kế chuyên dụng cho các luồng dữ liệu một chiều tốc độ cao và không định hướng không gian địa chỉ. Bằng việc loại bỏ hoàn toàn các kênh và tín hiệu quản lý địa chỉ, AXI4-Stream tối giản hóa tối đa cấu trúc bus để tập trung toàn bộ băng thông vật lý cho việc truyền tải luồng dữ liệu nối tiếp giữa Master và Slave [18].

Cơ chế điều luồng của AXI4-Stream dựa trên một chu kỳ bắt tay duy nhất thông qua hai tín hiệu điều khiển chủ đạo tích cực ở mức cao:

TVALID: Do Master kéo lên mức cao để báo hiệu rằng dữ liệu hiện tại đang hiện diện trên đường bus đã hợp lệ.

TREADY: Do Slave kéo lên mức cao để báo hiệu rằng nó đã sẵn sàng tiếp nhận dữ liệu từ bus.

Giao dịch truyền dữ liệu thực tế chỉ xảy ra tại sườn lên của xung nhịp khi cả hai tín hiệu TVALID và TREADY đồng thời ở trạng thái logic bằng 1 [18]. Kết hợp với tín hiệu TLAST dùng để đánh dấu phần tử cuối cùng của một block hoặc một gói dữ liệu, bus AXI4-Stream cho phép đẩy liên tục một Data Word trên mỗi chu kỳ xung nhịp hệ thống mà không gặp phải độ trễ do chuyển đổi địa chỉ. Nhờ đó, hệ thống khai thác được tối đa băng thông truyền tải giữa lõi DMA và khối xử lý tăng tốc phần cứng trong mạch PL [15], [18].

2.9. Tái chuẩn hóa và đánh giá

Sau khi mangle tính toán hoàn thành việc tích lũy dọc theo chiều sâu cấu trúc phân khối, các kết quả trung gian mang định dạng INT32 lưu trữ tại các hàng PE sẽ được đẩy tuần tự vào bộ nhớ đệm FWFT FIFO nhằm tối ưu hóa chu kỳ đọc không độ trễ [17]. Tại đây, module xử lý hậu kỳ thực hiện phép tính toán dịch bit làm tròn kết hợp mạch bão hòa nhằm ép kiểu dữ liệu từ INT32 quay trở lại định dạng INT8 [4]. Quá trình này được thực hiện thông qua bộ Rescale Pipeline phần cứng mô phỏng chính xác thuật toán dịch bit cố định của mạch RTL:

$$\text{Bias Rounding} = 1 \ll (\text{SCALE_SHIFT} - 1)$$

$$\text{Data}_{\text{shifted}} = (\text{Data}_{\text{int32}} + \text{Bias Rounding}) \gg \text{SCALE_SHIFT}$$

$$\text{Data}_{\text{int8}} = \text{clamp}(\text{Data}_{\text{shifted}} - 128, 127)$$

Hệ thống sau khi tích hợp toàn diện sẽ được tiến hành đánh giá hiệu năng dựa trên hai trục chỉ số chính:

1. **Hiệu năng tốc độ xử lý phần cứng:** Tiến hành đo đạc và so sánh trực tiếp thời gian thực thi phép nhân ma trận kích thước lớn khi chạy thuần túy bằng phần mềm trên CPU ARM Cortex-A53 (PS) so với thời gian xử lý khi kích hoạt bộ tăng tốc mangle tâm thu trên FPGA (PL) nhằm chứng minh gia tốc thông lượng. Đồng thời cũng đo các giá trị thời gian như Transfer Time, Compute Time để đánh giá tốc độ của các thành phần trong hệ thống.
2. **Kiểm chứng chức năng chính xác:** Toàn bộ tệp nhị phân đầu ra thu được từ kit KV260 sẽ được nạp ngược lại môi trường Python phần mềm. Tại đây, một mô hình toán học kiểm chứng chéo (Golden Model) được thiết lập bằng thư viện NumPy/PyTorch để thực hiện phép toán GEMM tương đương trên các ma trận gốc. Sau đó, thực hiện 2 việc: giải lượng tử kết quả để so sánh sai số với kết quả gốc FP32 và đối chiếu sai lệch bit.

Các chỉ số đánh giá sai số từ kết quả giải lượng tử hóa là Cosine Similarity, SQNR, MAE và RMSE (tương tự như các chỉ số khi đánh giá sai số của dữ liệu lượng tử hóa) nhằm kiểm tra nhiễu lượng tử và giữ được các đặc trưng của mô hình.

Việc đối chiếu trực tiếp không sai lệch (Bit-True Cross Verification) giữa ma trận phần cứng và ma trận phần mềm đảm bảo tính đúng đắn tuyệt đối của thiết kế mạch logic phần cứng trước khi tích hợp vào hệ thống lớn.

Chương II. KIẾN TRÚC HỆ THỐNG

Đồ án thiết kế và triển khai một bộ tăng tốc phần cứng chuyên dụng cho phép nhân ma trận trên nền tảng SoC AMD Kria KV260. Kiến trúc tổng thể của hệ thống là sự kết hợp chặt chẽ giữa hai thành phần: Hệ thống xử lý PS và Hệ thống logic lập trình được PL.

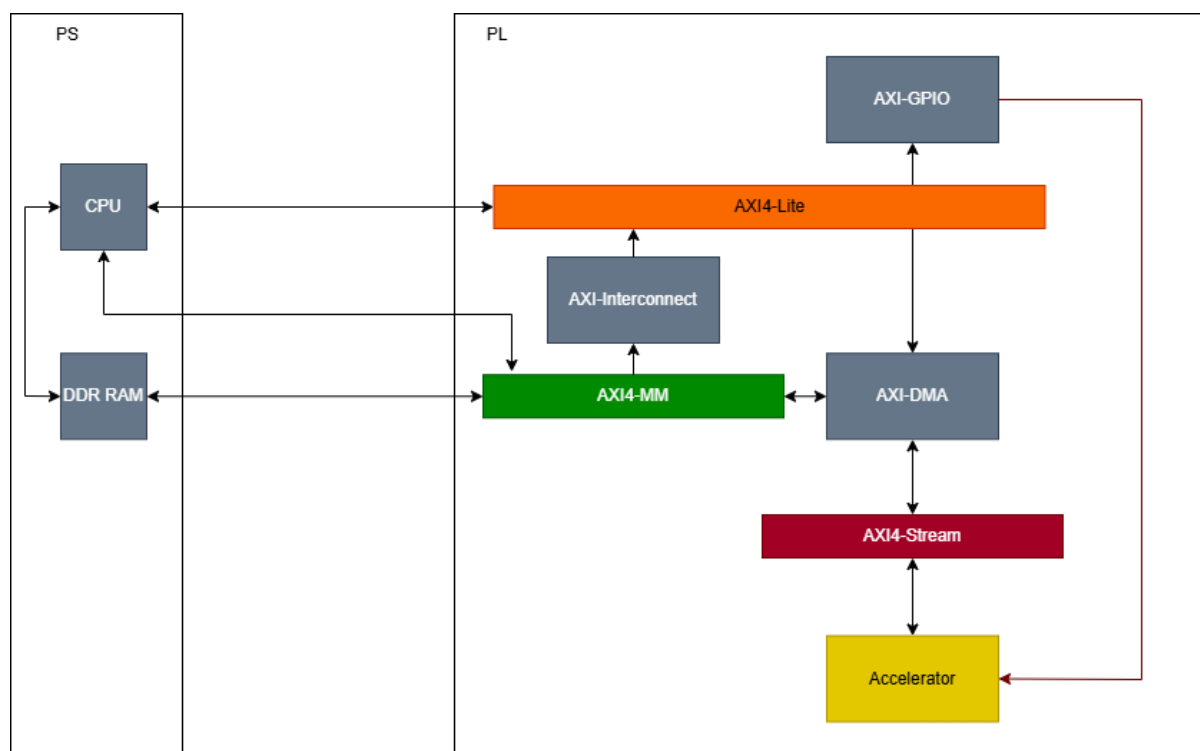
PS đóng vai trò điều khiển tổng thể, thực hiện cấu hình, chuẩn bị dữ liệu đầu vào và quản lý bộ nhớ. Ngược lại, PL thực hiện tính toán, chứa bộ tăng tốc cốt lõi được thiết kế bằng mã HDL nhằm khai thác tối đa khả năng xử lý song song.

Để hai phân hệ này giao tiếp với nhau với băng thông cao và tối ưu độ trễ, hệ thống phân tách rõ ràng các phân lớp giao tiếp thuộc kiến trúc AMBA AXI:

Giao thức AXI4-Lite được CPU sử dụng để truyền các lệnh cấu hình và kiểm tra trạng thái của các thanh ghi điều khiển bên trong các khối IP.

Giao thức AXI4 Memory-Mapped (AXI4-MM) đầy đủ được áp dụng cho tuyến đường tốc độ cao giữa AXI-DMA và bộ nhớ DDR nội bộ thuộc vùng PS. Giao thức này cho phép DMA đóng vai trò Master, chủ động thực hiện các phiên truyền gói dung lượng lớn dựa trên địa chỉ vùng nhớ được cấp phát.

Giao thức AXI4-Stream được sử dụng nội bộ trong vùng PL, đóng vai trò kết nối trực tiếp giữa AXI-DMA và bộ tăng tốc. Giao thức này truyền tải các dòng dữ liệu ma trận liên tục mà không cần kèm theo thông tin địa chỉ, giúp triệt tiêu hoàn toàn độ trễ quản lý và tối đa hóa thông lượng tính toán.



Hình 7. Tổng quan hệ thống

Toàn bộ các ma trận kích thước lớn ban đầu được nạp và lưu trữ trên bộ nhớ RAM DDR. CPU đóng vai trò quản lý điều khiển luồng cấp cao; tại đây, CPU thực hiện thuật toán phân chia ma trận lớn thành các khối ma trận con có kích thước phù hợp với dung lượng bộ đệm và SA xử lý ở PL. Đồng thời, CPU tính toán trước các hệ số định lượng và tham số tinh chỉnh tỉ lệ.

CPU sử dụng giao thức AXI4-Lite để ghi các thông số hình học của ma trận (như số vòng lặp tích lũy, kích thước khối) và các hệ số rescale vào thanh ghi chức năng của khối AXI-GPIO. Khối GPIO này sẽ giữ các mức tín hiệu điều khiển ổn định để thiết lập trạng thái vận hành cho bộ tăng tốc phần cứng trong PL.

Để bắt đầu chu kỳ tính toán, CPU ghi địa chỉ vật lý bắt đầu của các khối ma trận con A, B trên RAM DDR cùng số lượng byte cần dịch chuyển vào các thanh ghi lệnh của bộ điều khiển DMA thông qua bus AXI4-Lite. Ngay sau khi được kích hoạt, bộ điều khiển DMA chủ động gửi yêu cầu đến Interconnect thông qua bus AXI4-MM. DMA thực hiện đọc dữ liệu cấu trúc dữ liệu số nguyên INT8 liên tục từ RAM DDR theo chế độ truyền chuỗi mà không cần sự can thiệp của CPU, sau đó chuyển đổi sang chuẩn giao tiếp AXI4-Stream để đẩy trực tiếp vào bộ tăng tốc.

Sau khi bộ tăng tốc hoàn thành tính toán khối dữ liệu, kết quả đầu ra được đẩy ra dưới dạng luồng AXI4-Stream. Kênh thu của khối DMA tiếp nhận luồng dữ liệu này, đóng gói kèm địa chỉ đích vật lý đã cấu hình trước đó, và thực hiện ghi trực tiếp kết quả ngược trở lại RAM DDR thông qua bus AXI4-MM.

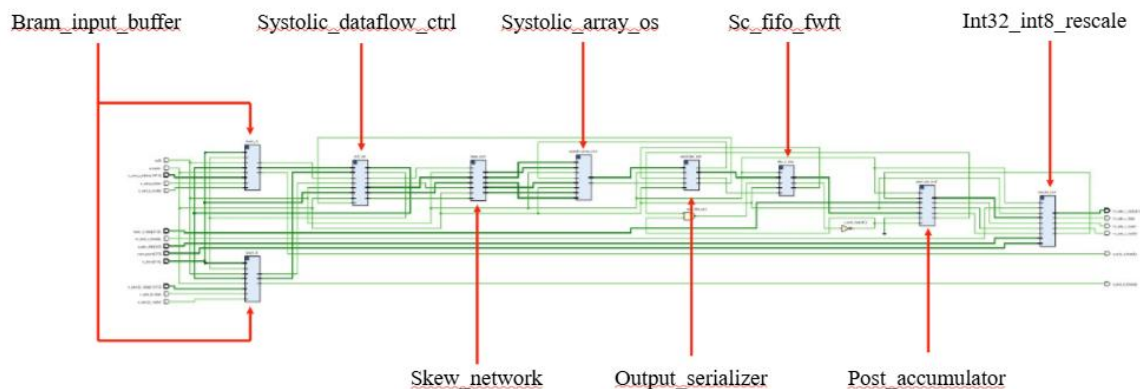
Ngay sau khi hoàn thành việc ghi toàn bộ khối ma trận kết quả vào RAM DDR, khối DMA sẽ phát một tín hiệu yêu cầu ngắt Interrupt Request (IRQ) tới CPU. Nhận được tín hiệu ngắt, CPU thực hiện thu thập khối ma trận con kết quả từ DDR để ghép vào vị trí tương ứng trên ma trận tổng thể, đồng thời giải phóng tài nguyên phần cứng và chuẩn bị cấu hình các tham số địa chỉ cho khối ma trận tiếp theo.

Chương III. THIẾT KẾ HỆ THỐNG PHẦN CỨNG.

1. Thiết kế bộ tăng tốc nhân ma trận.

1.1. Kiến trúc Top-level của bộ tăng tốc

Lõi của hệ thống là module `gemm_accelerator`, đóng vai trò là khối bao gói kết nối toàn bộ các phân hệ chức năng xử lý bên trong PL. Dữ liệu chảy qua bộ tăng tốc theo đường ống một chiều, được thiết kế chuyên biệt để tối đa hóa thông lượng tính toán và tận dụng hiệu quả tài nguyên phần cứng của chip Zynq UltraScale+.



Hình 8. Tổng quan kiến trúc của Top-level

Kiến trúc bên trong của bộ tăng tốc được chia thành 4 phần chính hoạt động đồng bộ với nhau:

1. Đệm dữ liệu đầu vào.
2. Điều khiển luồng và làm lệch dữ liệu.
3. Lõi tính toán.
4. Xử lý hậu kỳ.

1.2. Đệm dữ liệu đầu vào.

Dữ liệu từ khối AXI-DMA được đẩy vào bộ tăng tốc thông qua giao thức AXI4-Stream. Để giải quyết bài toán chênh lệch tốc độ giữa luồng truyền tải bus và tốc độ tiêu thụ dữ liệu của lõi tính toán, hệ thống sử dụng module `bram_input_buffer`.

Bộ đệm này được thiết kế dựa trên kỹ thuật Ping-Pong Buffer sử dụng các khối BRAM nội bộ của FPGA. Mỗi bộ đệm được chia thành hai phân vùng nhớ độc lập `ram_0` và `ram_1`. Trong quá trình hoạt động:

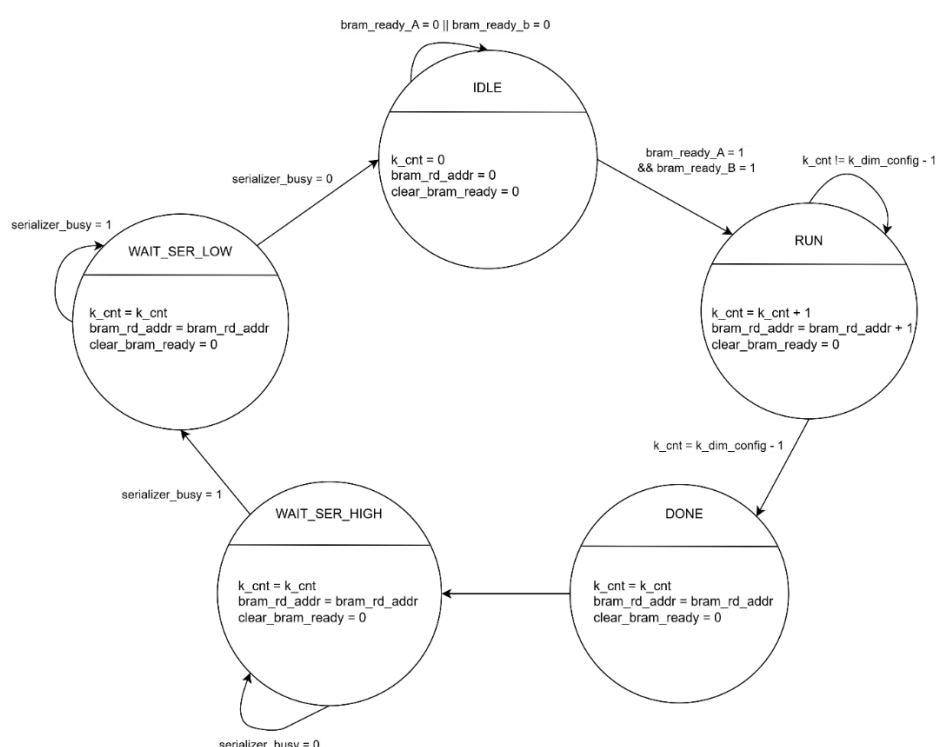
Con trỏ ghi: Tiếp nhận dữ liệu liên tục từ DMA và ghi vào một phân vùng.

Con trỏ đọc: Khối điều khiển SA sẽ hút dữ liệu ra từ phân vùng còn lại để đưa vào tính toán.

Khi cả hai quá trình đọc và ghi hoàn tất, module sẽ đảo trạng thái của 2 con trỏ `wr_bank` và `rd_bank`. Cơ chế này cho phép luồng dữ liệu tính toán được diễn ra liên tục, che giấu hoàn toàn độ trễ do quá trình truy xuất bộ nhớ DDR từ bên ngoài.

1.3. Điều khiển luồng

Là một máy trạng thái hữu hạn (Finite State Machine – FSM) chịu trách nhiệm tạo ra địa chỉ đọc tuần tự để quét qua bộ đệm BRAM dọc theo chiều K của khối ma trận. Nó đồng thời sinh ra các tín hiệu điều khiển quan trọng như cờ hợp lệ (`valid`), cờ xóa bộ cộng dồn (`clear_acc`) và cờ kết thúc khối (`last_mac`).



Hình 9. FSM của module `systolic_dataflow_ctrl`

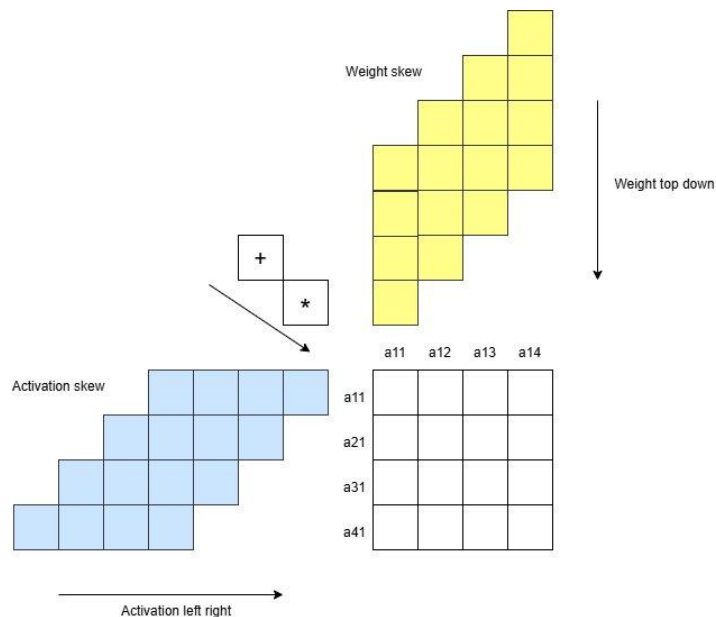
Trạng thái	Tên trạng thái	Chức năng chi tiết
3'd0	IDLE	Đứng chờ cho đến khi cả 2 khối BRAM A và B đều sẵn sàng dữ liệu (<code>bram_ready_A && bram_ready_B</code>). Khi thỏa mãn, reset bộ đếm và chuyển sang RUN.

3'd1	RUN	Giai đoạn đọc dữ liệu: Liên tục tăng địa chỉ BRAM (bram_rd_addr) và bộ đếm vòng lặp (k_cnt). Khi k_cnt đạt tới giá trị cấu hình (k_dim_config - 1), nó sẽ phát tín hiệu xóa cờ sẵn sàng (clear_bram_ready = 1) và chuyển sang DONE.
3'd2	DONE	Trạng thái trung gian, tự động chuyển ngay sang WAIT_SER_HIGH ở chu kỳ tiếp theo.
3'd3	WAIT_SER_HIGH	Chờ (Stall) cho đến khi bộ gom/xuất dữ liệu bắt đầu hoạt động và dừng cờ serializer_busy == 1, sau đó chuyển sang WAIT_SER_LOW.
3'd4	WAIT_SER_LOW	Tiếp tục chờ cho đến khi bộ tuần tự hóa xử lý xong và hạ cờ (!serializer_busy), lúc này hệ thống quay về IDLE để sẵn sàng cho lượt tính toán tiếp theo.

Bảng 1. Phân tích FSM systolic_dataflow_ctrl

1.4. Mạng làm lệch dữ liệu

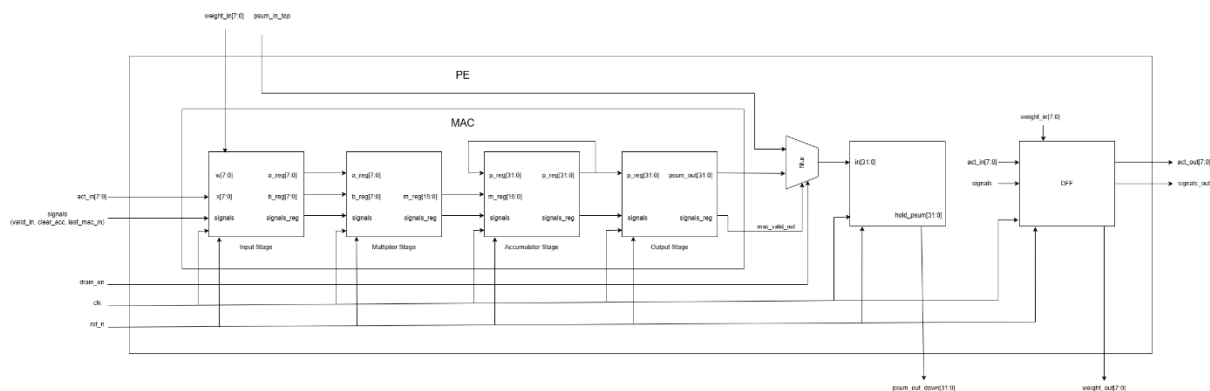
Đặc thù của kiến trúc SA là dữ liệu không được nạp vào toàn bộ mảng trong cùng một chu kỳ xung nhịp. Module skew_network sử dụng một mảng các thanh ghi dịch để tạo ra độ trễ hình bậc thang. Cụ thể, hàng/cột thứ i sẽ bị làm trễ i chu kỳ xung nhịp. Điều này đảm bảo dòng chảy của ma trận kích hoạt A và ma trận trọng số B nạp vào mảng theo dạng đường chéo, giúp các cặp dữ liệu tương ứng gặp nhau chính xác tại cùng một phần tử xử lý ở một thời điểm.



Hình 10. Minh họa skew network cho ma trận 4x4

1.5. Lỗi tính toán.

Đây là bộ phận cốt lõi tiêu tốn nhiều tài nguyên nhất và quyết định trực tiếp hiệu suất của toàn hệ thống, bao gồm 256 PEs được sắp xếp thành lưới tọa độ 16×16 .



Hình 11. Thiết kế cụ thể cho module *pe_wrapper*

Phần tử xử lý nhân cộng: Module *mac_core* thực hiện phép nhân hai toán hạng nguyên 8-bit (INT8) và cộng dồn vào thanh ghi chứa 32-bit (INT32). Lỗi MAC được thiết kế cực kỳ tối ưu bằng cách phân bổ các thanh ghi nội bộ (*a_reg*, *b_reg*, *m_reg*, *p_reg*) khớp chính xác với cấu trúc đường ống của các khối xử lý tín hiệu số DSP48E2 cứng trên FPGA Xilinx, giúp hệ thống đạt được xung nhịp hoạt động *Fmax* ở mức cao nhất.

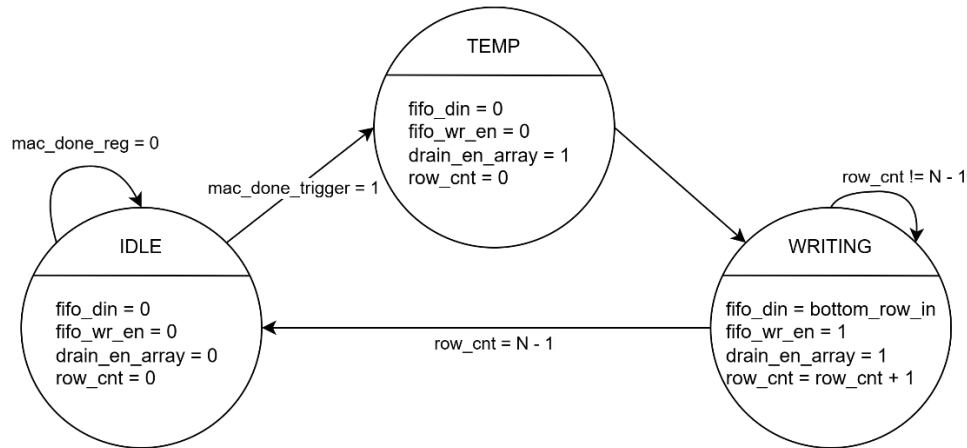
Luồng dữ liệu OS: Tại mỗi module *pe_wrapper*, kết quả cộng dồn được giữ nguyên tại chỗ trong suốt quá trình quét dọc theo chiều K. Ma trận Act trôi ngang từ trái sang phải, ma trận Weight trôi dọc từ trên xuống dưới. Thiết kế này giúp loại bỏ hoàn toàn các tác vụ đọc/ghi kết quả trung gian ra bộ nhớ ngoài, giảm mạnh mức tiêu thụ năng lượng. Khi tín hiệu *last_mac* xuất hiện, toàn bộ 16 hàng PE sẽ đồng loạt xả dữ liệu kết quả xuống hàng PE dưới cùng.

1.6. Xử lý hậu kỳ.

Do mảng tính toán nhả ra một khối lượng dữ liệu khổng lồ cùng lúc, phân hệ này làm nhiệm vụ thu thập, tích lũy và chuẩn hóa định dạng dữ liệu trước khi DMA thu về RAM.

1.6.1. Khối tuần tự hóa và FIFO đệm.

Module *output_serializer* chụp lấy dải dữ liệu song song rộng 512-bit từ hàng dưới cùng của Systolic Array, tương ứng với 16 phần tử kết quả INT32 được xuất đồng thời. Do bus AXI4-Stream phía sau chỉ xử lý dữ liệu theo dạng tuần tự, module này thực hiện tách tuần tự từng phần tử INT32 và đẩy vào bộ đệm FIFO.



Hình 12. FSM của module output_serializer

Trạng thái	Tên trạng thái	Chức năng chi tiết
2'd0	IDLE	Đứng chờ. Khi có tín hiệu kích hoạt (<code>mac_done_trigger == 1</code>), hệ thống chuẩn bị chuyển trạng thái. Các cờ dữ liệu vẫn ở mức 0.
2'd1	TEMP (hoặc SETUP)	Trạng thái đệm ấn (1 chu kỳ). Bắt đầu bật cờ <code>drain_en_array = 1</code> để kéo dữ liệu từ mảng tâm thu xuống. Tuy nhiên, ép cờ <code>fifo_wr_en = 0</code> để tạo độ trễ 1 nhịp xung nhịp, ngăn FIFO ghi lặp dữ liệu cũ của hàng cuối cùng.
2'd2	WRITING	Bắt đầu chuỗi xả dữ liệu. Bật cờ <code>fifo_wr_en = 1</code> để chính thức ghi vào FIFO. Tăng dần bộ đếm <code>row_cnt</code> . Khi xả đủ N hàng, FSM hạ các cờ điều khiển và quay lại IDLE.

Bảng 2. Phân tích FSM output_serializer

Mặc dù về mặt thuật toán logic, hệ thống trải qua 3 giai đoạn hành vi (Chờ → Tạo trễ → Ghi) như bảng trên; nhưng trong mã nguồn Verilog thực tế, trạng thái TEMP đã được ẩn vào quá trình chuyển dịch cấu trúc từ IDLE sang WRITING. Việc gộp này tận dụng chính 1 chu kỳ trễ tự nhiên khi lật thanh ghi trạng thái phần cứng. Kỹ thuật này giúp FSM thực tế trong code chỉ cần 2 trạng thái (1-bit thanh ghi state) thay vì 3 trạng thái (2-bit thanh ghi state), tối ưu hóa tối đa tài nguyên cổng logic (LUTs/Registers) trên FPGA mà vẫn đảm bảo chính xác 100% hành vi thời gian.

FIFO được xây dựng bằng Hard IP xpm_fifo_sync ở chế độ FWFT, cho phép dữ liệu đầu tiên xuất hiện ngay tại ngõ ra mà không cần thêm chu kỳ đọc phụ, từ đó giảm độ trễ toàn hệ thống.

1.6.2. Cộng dồn khối lớn.

Để hỗ trợ phép nhân ma trận có chiều sâu K lớn hơn kích thước phần cứng của SA, hệ thống áp dụng kỹ thuật chia ma trận thành nhiều khối theo chiều K (K-tiling). Mỗi lần quét chỉ tạo ra một kết quả tích lũy tạm thời:

$$C = \sum_{t=0}^{T-1} X_i \cdot W_i$$

Trong đó:

X_i và W_i là các tile tại lần quét thứ t

$T = \text{num_k_tiles}$

Module post_accumulator duy trì một vùng RAM nội bộ để lưu các tổng trung gian. Sau mỗi lần SA hoàn tất một tile, kết quả INT32 mới sẽ được cộng dồn với giá trị trước đó:

$$C_{acc}^{t+1} = C_{acc}^t + C_t$$

Khi bộ đếm đạt đủ số lượng tile được cấu hình bởi phần mềm, tổng INT32 cuối cùng mới được xuất sang khối tái chuẩn hóa.

1.6.3. Tái chuẩn hóa lượng tử.

Sau quá trình MAC và cộng dồn nhiều tile, giá trị đầu ra tồn tại dưới dạng số nguyên có dấu 32-bit. Nếu truyền trực tiếp dữ liệu INT32 về DDR RAM:

Băng thông AXI DMA tăng gấp 4 lần so với INT8

Dung lượng bộ nhớ tiêu thụ lớn

Giảm hiệu quả xử lý của toàn hệ thống

Do đó, hệ thống thực hiện tái chuẩn hóa (requantization) để đưa dữ liệu trở lại miền INT8 trước khi ghi về bộ nhớ.

Module int32_int8_rescale thực hiện phép lượng tử hóa đối xứng bằng phần cứng theo công thức:

$$q = \text{clip}\left(\frac{x}{2^8} + z_p\right)$$

Trong đó:

x : giá trị INT32 sau cộng dồn

s : hệ số dịch bit `scale_shift`

z_p : giá trị bù `zero_point`

`clip(.)`: hàm xén biên về miền INT8

Trong phần cứng, phép chia được thay bằng phép dịch phải số học:

$$q = \text{clip}((x \gg s) + z_p)$$

Do phép dịch phải số học tương đương phép chia theo lũy thừa của 2:

$$x \gg s \approx \frac{x}{2^s}$$

Cách tiếp cận này giúp tránh sử dụng bộ chia phần cứng vốn tiêu tốn LUT và có độ trễ lớn.

Tham số `shift_val` quyết định mức độ thu nhỏ của dữ liệu INT32 trước khi ép kiểu về INT8. Trong phần cứng, phép chia được thay bằng phép dịch phải số học: $q = x \gg \text{shift_val}$. Do đó, `shift_val` càng nhỏ thì dữ liệu sau chuẩn hóa càng lớn và ngược lại.

Nếu giá trị dịch quá nhỏ, nhiều phần tử sẽ vượt miền biểu diễn INT8 và bị mạch xén biên ép về ± 127 . Hiện tượng này gây bão hòa dữ liệu, làm mất động học giá trị và suy giảm độ chính xác inference.

Ngược lại, nếu giá trị dịch quá lớn, phần lớn kết quả sau chuẩn hóa sẽ tiến gần về 0. Khi đó dữ liệu mất đi các đặc trưng quan trọng do sai số lượng tử hóa quá lớn.

Qua quá trình mô phỏng và thực nghiệm với tập dữ liệu của hệ thống, giá trị `shift_val = 10` được lựa chọn vì tạo ra sự cân bằng hợp lý giữa hạn chế hiện tượng bão hòa, giữ được độ phân giải số học, giảm mất mát thông tin sau lượng tử hóa và tối ưu băng thông lưu trữ khi chuyển từ INT32 về INT8. Ngoài ra, do $2^{10} = 1024$ nên phép tái chuẩn hóa tương đương với việc chia dữ liệu cho 1024 bằng phần cứng chỉ với một phép dịch bit, giúp giảm đáng kể tài nguyên logic so với sử dụng bộ chia số học.

Sau bước `scale` và cộng `zero_point`, kết quả tiếp tục đi qua mạch xén biên:

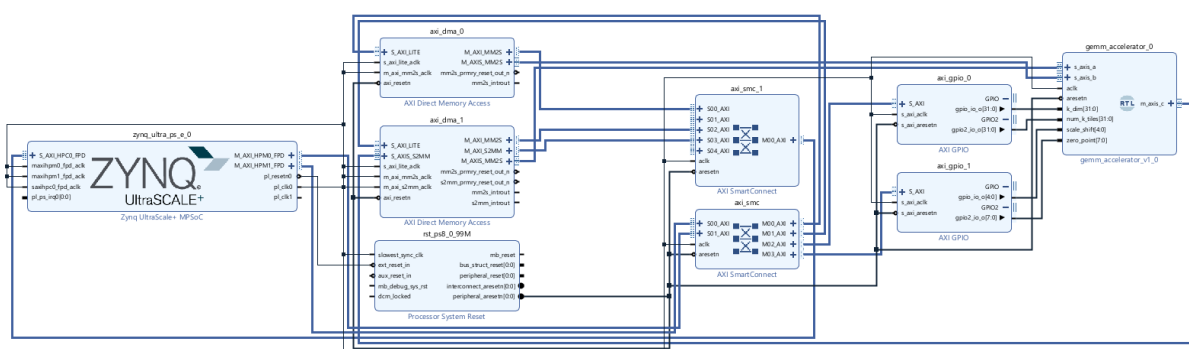
$$q_{final} = \begin{cases} 127 & q > 127 \\ -128 & q < -128 \\ q & otherwise \end{cases}$$

2. Triển khai hệ thống.

CPU và bộ tăng tốc giao tiếp với nhau thông qua kiến trúc bus AXI. Giao thức AXI4-Lite được sử dụng để truyền các lệnh cấu hình và tham số điều khiển 32-bit, trong đó CPU đóng vai trò AXI Master còn các ngoại vi trong PL đóng vai trò AXI Slave. Các tham số được truyền thông qua AXI GPIO bao gồm: `k_dim`, `num_k_tiles`, `scale_shift`, `zero_point`

Các khối AXI SmartConnect được sử dụng để thực hiện định tuyến, đồng bộ và phân xử truy cập bus giữa PS, DMA và các ngoại vi trong PL. Tín hiệu clock và reset của toàn bộ hệ thống được cung cấp từ khối PS và phân phối tới các module trong PL thông qua hạ tầng thiết kế của Vivado.

Toàn bộ hệ thống hoạt động đồng bộ tại tần số clock 100 MHz.



Hình 13. Hệ thống chi tiết được triển khai trong Vivado



Hình 14. Cây thư mục của toàn bộ hệ thống

3. Kiểm tra và phân tích.

3.1. Kiểm tra chức năng

Để kiểm tra tính chính xác của bộ tăng tốc, hệ thống được mô phỏng bằng testbench Register-Transfer Level (RTL) trên Vivado Simulator theo phương pháp end-to-end verification. Testbench thực hiện mô phỏng đầy đủ luồng dữ liệu từ DMA đầu vào, bộ tăng tốc General Matrix Multiply (GEMM) cho đến DMA đầu ra tương tự như khi hệ thống hoạt động thực tế trên FPGA.

Dữ liệu đầu vào được sinh ngẫu nhiên dưới định dạng số nguyên có dấu INT8 trong miền giá trị $[-128, 127]$. Hai ma trận: $A \in \mathbb{Z}^{16 \times 512}$ và $B \in \mathbb{Z}^{512 \times 16}$ được chia thành nhiều tile theo chiều K để truyền tuần tự vào bộ tăng tốc thông qua giao thức AXI4-Stream.

Đối với mỗi phần tử đầu ra, testbench đồng thời xây dựng một mô hình tham chiếu bằng phần mềm (golden model) để thực hiện: phép nhân cộng INT32, cộng dồn nhiều k-tile, tái chuẩn hóa về INT8, xén biên theo đúng thuật toán phần cứng.

Kết quả từ phần cứng sau khi xuất qua AXI4-Stream được lưu lại và so sánh trực tiếp với golden model. Hệ thống chỉ được xem là hợp lệ khi toàn bộ phần tử đầu ra khớp hoàn toàn với kết quả tham chiếu.

Kết quả mô phỏng console được thể hiện như sau:

[150000 ns] GEMM ACCELERATOR TESTBENCH

N=16 | K_DIM=32 | K_BLOCKS=2 | NUM_K_TILES=16 | K_TOTAL=512

SCALE_SHIFT=10 | ZERO_POINT=0

[150000 ns] [SW] Tính golden C = A x B...

[150000 ns] [SW] Golden model hoàn thành.

[150000 ns] [RX] Waiting for output matrix C...

[150000 ns] [TX] Bắt đầu gửi dữ liệu (16 tiles)...

[150000 ns] [TX] Tile 1 / 16

[490000 ns] [TX] Tile 2 / 16

[830000 ns] [TX] Tile 3 / 16

[1170000 ns] [TX] Tile 4 / 16

[2020000 ns] [TX] Tile 5 / 16

[2880000 ns] [TX] Tile 6 / 16

[3740000 ns] [TX] Tile 7 / 16

[4600000 ns] [TX] Tile 8 / 16

[5460000 ns] [TX] Tile 9 / 16

[6320000 ns] [TX] Tile 10 / 16

[7180000 ns] [TX] Tile 11 / 16

[8040000 ns] [TX] Tile 12 / 16

[8900000 ns] [TX] Tile 13 / 16

[9760000 ns] [TX] Tile 14 / 16

[10620000 ns] [TX] Tile 15 / 16

[11480000 ns] [TX] Tile 16 / 16

[12340000 ns] [TX] Đã nạp đủ dữ liệu. Cho kết quả...

[14296000 ns] [RX] SOF nhận được - bắt đầu ghi matrix C...

[14446000 ns] [RX] Nhận đủ 16 hàng.

GOLDEN MODEL COMPARISON

C_npu_result (INT8) - Hardware Output:

```
Row[ 0]: 64 68 63 -128 -118 100 -115 12 127 126 127 127 117 46 -128 -71
Row[ 1]: 51 -72 118 -1 127 -26 -43 -128 79 51 -99 -44 127 107 2 -22
Row[ 2]: 2 31 127 -18 -128 3 -74 19 -128 127 61 59 -128 -40 -109 -90
Row[ 3]: -50 3 77 127 17 14 14 -128 -8 58 -45 -128 71 45 -28 92
Row[ 4]: -83 127 -77 58 35 28 -75 -52 -41 -46 -128 -128 -2 115 62 -50
Row[ 5]: 109 76 -128 -128 127 127 -70 108 55 103 -13 -62 8 53 -103 127
Row[ 6]: -128 -128 -22 -128 -27 127 95 127 -65 -50 -41 127 -66 -18 127 -90
Row[ 7]: 100 -22 27 -71 -61 53 -66 22 127 -109 127 88 2 30 -126 -69
Row[ 8]: 62 9 -124 25 47 -19 -35 -76 -128 8 -128 -128 127 54 5 80
Row[ 9]: -108 74 -117 -81 107 -68 126 -128 127 81 3 -92 10 -81 -119 127
Row[10]: 127 127 -71 34 -17 -5 104 -7 -91 127 39 127 -34 127 -128 -108
Row[11]: 40 -128 26 127 19 98 -6 -128 -92 64 -86 127 -128 -20 127 108
Row[12]: 111 -59 127 -103 -50 -128 106 59 127 -128 32 -128 -76 -128 -19 98
Row[13]: 127 -47 127 -11 83 -11 -76 -128 84 127 -34 127 51 -128 -94 107
```

Row[14]:	-128	71	127	127	87	7	16	-12	127	-128	-128	-8	101	-128	100	85
Row[15]:	-95	61	127	-20	41	106	121	127	73	39	67	127	-43	-10	47	-128

C_expected (INT8) - Software Golden:																

Row[0]:	64	68	63	-128	-118	100	-115	12	127	126	127	127	117	46	-128	-71
Row[1]:	51	-72	118	-1	127	-26	-43	-128	79	51	-99	-44	127	107	2	-22
Row[2]:	2	31	127	-18	-128	3	-74	19	-128	127	61	59	-128	-40	-109	-90
Row[3]:	-50	3	77	127	17	14	14	-128	-8	58	-45	-128	71	45	-28	92
Row[4]:	-83	127	-77	58	35	28	-75	-52	-41	-46	-128	-128	-2	115	62	-50
Row[5]:	109	76	-128	-128	127	127	-70	108	55	103	-13	-62	8	53	-103	127
Row[6]:	-128	-128	-22	-128	-27	127	95	127	-65	-50	-41	127	-66	-18	127	-90
Row[7]:	100	-22	27	-71	-61	53	-66	22	127	-109	127	88	2	30	-126	-69
Row[8]:	62	9	-124	25	47	-19	-35	-76	-128	8	-128	-128	127	54	5	80
Row[9]:	-108	74	-117	-81	107	-68	126	-128	127	81	3	-92	10	-81	-119	127
Row[10]:	127	127	-71	34	-17	-5	104	-7	-91	127	39	127	-34	127	-128	-108
Row[11]:	40	-128	26	127	19	98	-6	-128	-92	64	-86	127	-128	-20	127	108
Row[12]:	111	-59	127	-103	-50	-128	106	59	127	-128	32	-128	-76	-128	-19	98
Row[13]:	127	-47	127	-11	83	-11	-76	-128	84	127	-34	127	51	-128	-94	107
Row[14]:	-128	71	127	127	87	7	16	-12	127	-128	-128	-8	101	-128	100	85
Row[15]:	-95	61	127	-20	41	106	121	127	73	39	67	127	-43	-10	47	-128
=====																
TEST SUMMARY																
=====																
K_DIM=32 NUM_K_TILES=16 K_TOTAL=512																
SCALE_SHIFT=10 ZERO_POINT=0																
SOF (tuser) OK : YES																
EOF (tlast) OK : YES																

[PASS] TAT CA DUNG: 256 / 256 pixels!																
=====																

Bảng 3. Kết quả hiển thị trên Console

Kết quả này cho thấy:

Toàn bộ 256 phần tử của ma trận đầu ra 16×16 đều chính xác,

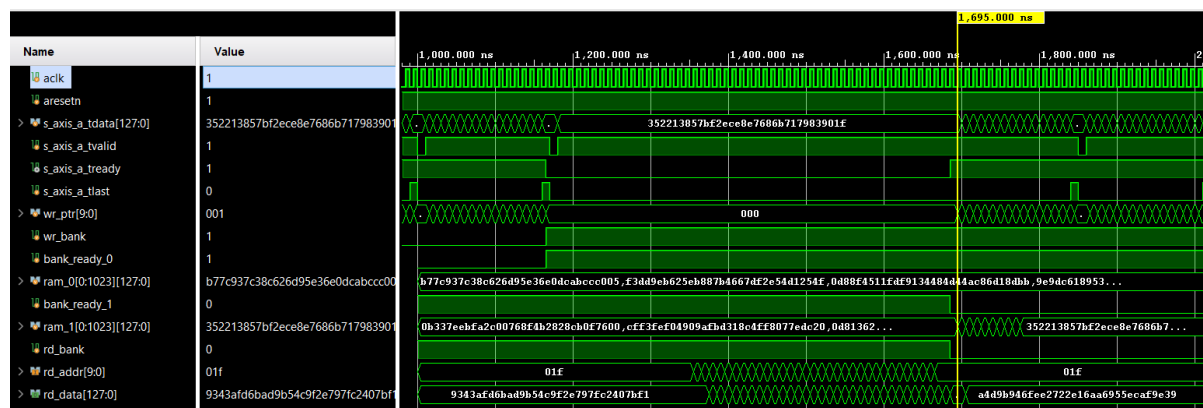
Kiến trúc Systolic Array hoạt động đúng,

Cơ chế cộng dồn nhiều K-tile chính xác,

Thuật toán requantization $\text{INT32} \rightarrow \text{INT8}$ hoạt động đúng với tham số $\text{scale_shift} = 10$,

Giao thức AXI4-Stream giữa các khối hoạt động ổn định.

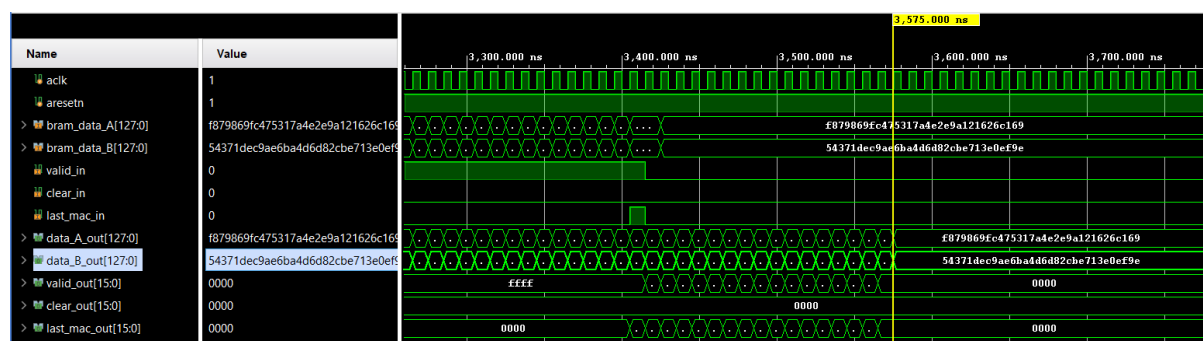
Trong quá trình mô phỏng, testbench cũng ghi nhận thời điểm bộ tăng tốc bắt đầu xuất dữ liệu kết quả thông qua tín hiệu tuser, cho phép xác nhận hoạt động đồng bộ giữa pipeline tính toán và khối DMA đầu ra.



Hình 15. Waveform ghi dữ liệu từ AXI DMA vào bộ đệm BRAM

Hình 15 thể hiện quá trình nhận dữ liệu từ AXI DMA vào bộ đệm BRAM của ma trận A. Khi tín hiệu tvalid và tready đồng thời ở mức ‘1’, dữ liệu trên bus tdata được chấp nhận và ghi vào BRAM sau một chu kỳ clock theo cơ chế synchronous write của FPGA. Trong quá trình này, BRAM vẫn đồng thời thực hiện đọc dữ liệu từ bank còn lại nhằm phục vụ tính toán song song của systolic array.

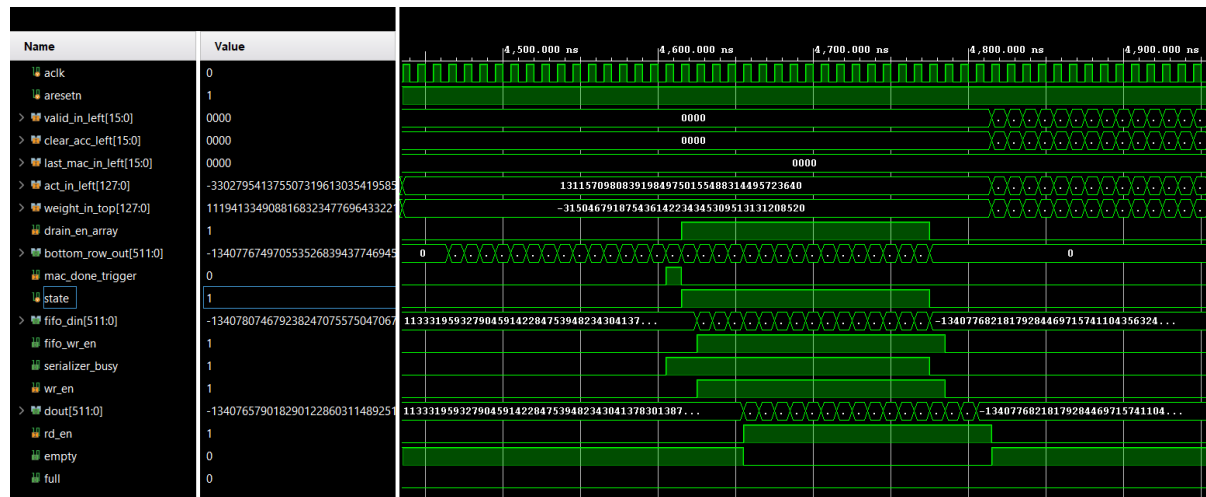
Ngoài ra, khi cả hai bank của ping-pong buffer đều ở trạng thái đầy (bank_ready = 1), tín hiệu tready sẽ tự động chuyển xuống mức ‘0’ để tạm dừng quá trình truyền dữ liệu từ DMA. Kết quả waveform cho thấy cơ chế AXI-Stream handshake và ping-pong buffer hoạt động chính xác theo thiết kế.



Hình 16. Waveform skew dữ liệu nhận từ BRAM

Hình 16 thể hiện waveform của khối systolic_dataflow_ctrl và skew_network trong quá trình đồng bộ dữ liệu đầu vào cho systolic array. Có thể thấy dữ liệu data_A_out và data_B_out được tạo độ trễ tuần tự trước khi đưa vào mảng PE, với độ trễ tối đa 16 chu kỳ clock tương ứng với số phần tử trong một hàng/cột của ma trận 16×16.

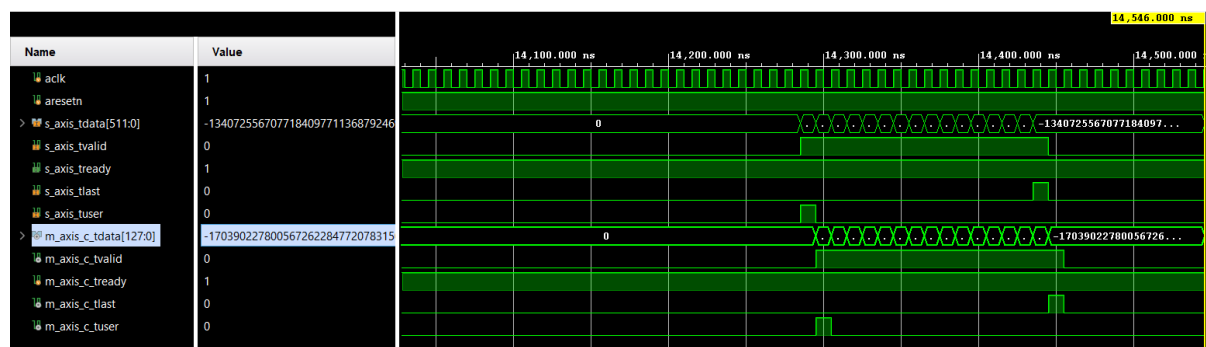
Ngoài dữ liệu, các tín hiệu điều khiển như `valid_out`, `clear_out` và `last_mac_out` cũng được dịch tuần tự thông qua các thanh ghi dịch nhằm đảm bảo dữ liệu và tín hiệu điều khiển luôn đồng bộ theo từng chu kỳ xử lý của systolic array. Kết quả waveform cho thấy cơ chế skew dữ liệu và đồng bộ pipeline hoạt động chính xác theo thiết kế.



Hình 17. Waveform tín hiệu điều khiển SA và lưu kết quả vào FIFO

Hình 17 thể hiện waveform điều khiển quá trình xuất dữ liệu từ systolic array và lưu kết quả vào FIFO thông qua khối `output_serializer`. Sau khi quá trình MAC hoàn tất, tín hiệu `mac_done_trigger` được kích hoạt đồng thời với trạng thái `serializer_busy = 1`, cho phép tín hiệu `drain_en_array` được bật để bắt đầu quá trình drain dữ liệu khỏi systolic array.

Đồng thời, FSM bên trong `output_serializer` chuyển trạng thái từ IDLE sang WRITING để ghi dữ liệu kết quả vào FIFO. Khoảng thời gian giữa tín hiệu `wr_en = 1` và `rd_en = 1` cho thấy FIFO hoạt động đúng theo cơ chế FWFT (First-Word Fall-Through), trong đó dữ liệu đầu tiên sẽ tự động xuất hiện ở ngõ ra mà không cần thêm chu kỳ đọc bổ sung. Kết quả waveform xác nhận quá trình điều khiển xuất dữ liệu và lưu trữ FIFO hoạt động chính xác theo thiết kế.



Hình 18. Waveform tái chuẩn hóa dữ liệu và truyền AXI-Stream output

Hình 18 thể hiện waveform của khối Rescale và quá trình truyền dữ liệu ra giao diện AXI-Stream output. Có thể thấy dữ liệu đầu ra và các tín hiệu điều khiển bị trễ một chu kỳ clock do khối Rescale cần thực hiện quá trình tái chuẩn hóa dữ liệu từ INT32 về INT8 trước khi xuất ra ngoài.

Sau khi quá trình rescale hoàn tất, dữ liệu sẽ được truyền thông qua giao diện AXI-Stream khi tín hiệu tvalid và tready đồng thời ở mức '1'. Kết quả waveform cho thấy quá trình tái chuẩn hóa và truyền dữ liệu output hoạt động chính xác theo thiết kế.

3.2. Kiểm tra timing

Sau khi xác nhận tính đúng đắn chức năng ở mức RTL bằng behavioral simulation, hệ thống tiếp tục được kiểm tra timing nhằm đánh giá khả năng hoạt động ổn định của thiết kế sau quá trình tổng hợp và triển khai vật lý trên FPGA.

Quá trình kiểm tra timing được thực hiện bằng các bước: Synthesis → Implementation → Static Timing Analysis (STA) trong môi trường Vivado Design Suite với xung nhịp $f_{clk} = 100\text{MHz}$ tương ứng với $T_{clk} = 10\text{ns}$.

Thông số	Giá trị
Worst Negative Slack (WNS)	2.238ns
Total Negative Slack (TNS)	0 ns
Worst Hold Slack (WHS)	0.010 ns
Total Hold Slack (THS)	0 ns
Clock Frequency	100 MHz

Bảng 4. Kết quả Static Timing Analysis sau Implementation

Kết quả cho thấy $WNS = 2.238\text{ns} > 0$ đồng nghĩa toàn bộ các đường dữ liệu trong thiết kế đều đáp ứng yêu cầu setup timing với khoảng dư thời gian lớn hơn 2ns so với chu kỳ clock mục tiêu.

Do đó, độ trễ lớn nhất của đường dữ liệu tới hạn có thể được xác định là $T_{critical} = 10 - 2.238 = 7.762\text{ns}$. Suy ra tần số hoạt động cực đại lý thuyết:

$$F_{max} \approx \frac{1}{7.762\text{ns}} \approx 128.8\text{MHz}$$

Bên cạnh đó $TNS = 0\text{ns}$ xác nhận không tồn tại endpoint nào vi phạm setup timing trong toàn bộ hệ thống. Đồng thời $THS = 0\text{ns}$ và $WHS = 0.010\text{ns} > 0$ cho thấy thiết kế cũng không gặp lỗi hold timing sau quá trình Placement & Routing vật lý.

Kết quả timing đạt được nhờ kiến trúc pipeline được tối ưu theo cấu trúc DSP48E2 của FPGA UltraScale+. Trong mỗi phần tử xử lý PE, module MAC sử dụng các tầng thanh ghi bao gồm: a_reg, b_reg, m_reg, p_reg. Cách tổ chức này giúp giảm

chiều dài logic tổ hợp trên mỗi chu kỳ clock, tăng khả năng retiming của công cụ Vivado và tận dụng tối đa các khối DSP cứng trên FPGA.

Mặc dù hệ thống sử dụng tới 256 DSP hoạt động đồng thời trong mảng Systolic Array kích thước 16×16 , thiết kế vẫn đáp ứng timing ổn định tại 100MHz mà không xuất hiện critical path nghiêm trọng giữa các PE hoặc khối cộng dồn INT32. Kết quả trên chứng minh kiến trúc pipeline của bộ tăng tốc đã được tổ chức hợp lý, đảm bảo hệ thống hoạt động ổn định sau triển khai vật lý trên FPGA.

3.3. Phân tích PPA của hệ thống.

Sau khi hoàn tất kiểm tra chức năng và timing, hệ thống tiếp tục được đánh giá theo các tiêu chí PPA (Performance – Power – Area) nhằm xác định khả năng đáp ứng hiệu năng, mức sử dụng tài nguyên phần cứng, và hiệu quả năng lượng của bộ tăng tốc GEMM trên nền tảng FPGA AMD Kria KV260.

Về mặt hiệu năng (Performance), hệ thống đã được đánh giá thông qua phân tích timing ở Mục 3.2 với kết quả: không xuất hiện vi phạm timing, đạt $WNS = 2.238ns$ và đáp ứng ổn định tại tần số hoạt động 100MHz.

Phần này tập trung phân tích hai tiêu chí còn lại của PPA gồm: Area (tài nguyên phần cứng), Power (công suất tiêu thụ).

3.3.1. Phân tích tài nguyên phần cứng.

Tài nguyên	Sử dụng	Khả dụng	Mức sử dụng (%)
LUT	28622	117120	22.44
LUTRAM	11171	57600	19.39
FF	35690	234240	15.24
BRAM	34	144	23.61
DSP	256	1248	20.51

Bảng 5. Mức sử dụng tài nguyên FPGA sau Implementation

Ngoài thống kê tổng quát, Vivado cũng cho phép phân tích chi tiết mức sử dụng tài nguyên theo từng module trong hệ thống. Kết quả được thể hiện ở Hình 19.

Name	CLB LUTs (117120)	CLB Registers (234240)	CARRY8 (14640)	F7 Muxes (58560)	CLB (14640)	LUT as Logic (117120)	LUT as Memory (57600)	Block RAM Tile (144)	DSPs (1248)	BUFG_PS (96)	PS8 (1)
gemm_block_wrapper	28622	35690	358	1026	6303	17451	11171	34	256	1	1
gemm_block_i (gemm_block)	28622	35690	358	1026	6303	17451	11171	34	256	1	1
axi_dma_0 (gemm_block_axi)	693	984	9	0	197	641	52	8.5	0	0	0
axi_dma_1 (gemm_block_axi)	2014	3066	23	0	516	1879	135	11	0	0	0
axi_gpio_0 (gemm_block_axi)	63	222	0	0	32	63	0	0	0	0	0
axi_gpio_1 (gemm_block_axi)	43	74	0	0	17	43	0	0	0	0	0
axi_smc (gemm_block_axi_sr)	1812	1788	10	2	363	1805	7	0	0	0	0
axi_smc_1 (gemm_block_axi)	2668	4806	0	0	621	1983	685	0	0	0	0
gemm_accelerator_0 (gemm_block)	21321	24717	316	1024	4803	11030	10291	14.5	256	0	0
inst (gemm_block_gemm)	21321	24717	316	1024	4803	11030	10291	14.5	256	0	0
bram_A (gemm_block)	5948	61	2	512	901	1212	4736	0	0	0	0
bram_B (gemm_block)	5938	57	2	512	927	1202	4736	0	0	0	0
ctrl_inst (gemm_block)	163	103	14	0	74	163	0	0	0	0	0
fifo_C_inst (gemm_block)	55	51	64	0	83	55	0	14.5	0	0	0
post_acc_inst (gemm_block)	3904	572	234	0	716	3386	518	0	0	0	0
rescale_inst (gemm_block)	18	130	0	0	48	18	0	0	0	0	0
serializer_inst (gemm_block)	11	524	0	0	149	11	0	0	0	0	0
skew_inst (gemm_block)	297	0	0	0	72	12	285	0	0	0	0
systolic_array_inst (gemm_block)	5014	23219	0	0	2794	4998	16	0	256	0	0

Hình 19. Phân bố tài nguyên theo từng module trong hệ thống

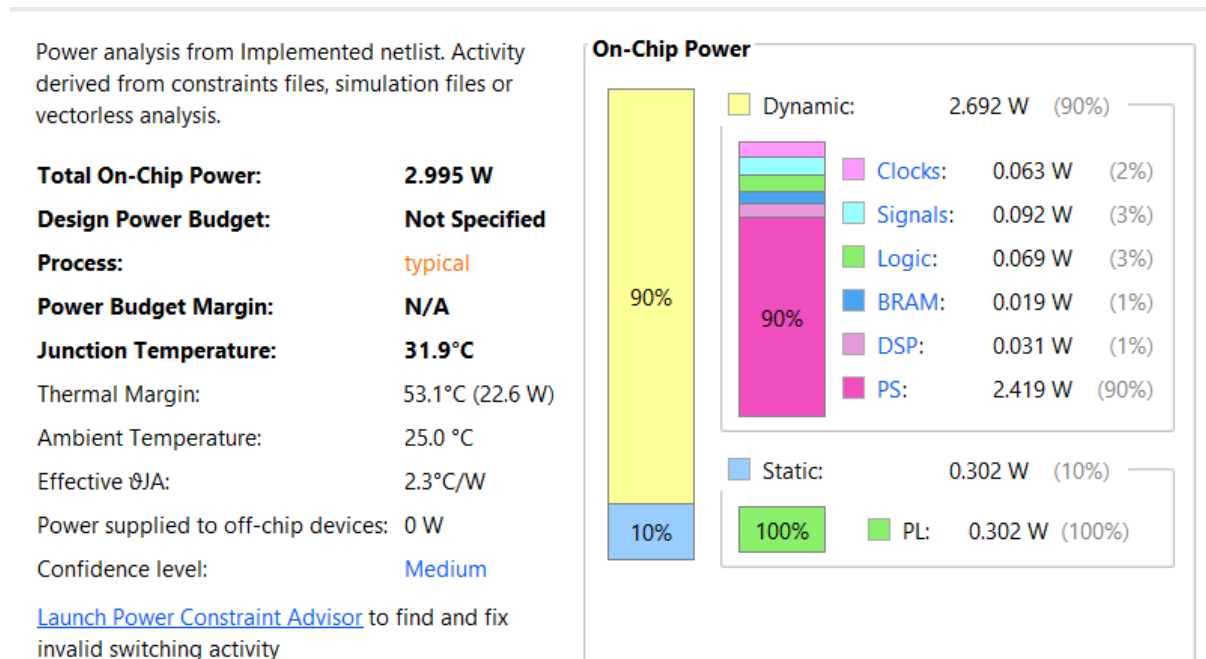
Kết quả cho thấy phần lớn tài nguyên phần cứng tập trung tại khối `gemm_accelerator`, đặc biệt là `systolic_array_inst`, `bram_A`, `bram_B` và `post_acc_inst`. Trong đó, `systolic_array_inst` sử dụng 256 DSP của hệ thống, đồng thời tiêu thụ lượng lớn thanh ghi FF nhằm xây dựng pipeline cho các phần tử xử lý PE. Điều này hoàn toàn phù hợp với đặc điểm của kiến trúc Systolic Array. Mỗi PE thực hiện một phép nhân cộng INT8 độc lập và được ánh xạ trực tiếp lên DSP48E2 của FPGA. Với cấu hình mảng 16×16 , hệ thống cần 256 khối DSP hoạt động song song.

Ngoài DSP, lượng lớn thanh ghi FF trong `systolic_array_inst` và `serializer_inst` được sử dụng để xây dựng pipeline dữ liệu, đồng bộ tín hiệu valid và duy trì luồng truyền dữ liệu liên tục giữa các PE. Hai khối `bram_A` và `bram_B` chiếm phần lớn Look-up Table (LUT) Memory của thiết kế do triển khai cơ chế Ping-Pong Buffer bằng BRAM nội bộ FPGA. Khối `post_acc_inst` sử dụng đáng kể LUT logic và Carry Chain do phải thực hiện cộng dồn INT32 nhiều tile theo chiều K, lưu trữ kết quả trung gian, và xử lý pipeline cộng tích lũy. Trong khi đó, các module `ctrl_inst`, `rescale_inst`, `skew_inst` chỉ chiếm tỷ lệ tài nguyên nhỏ do chủ yếu bao gồm FSM điều khiển, thanh ghi dịch và logic tổ hợp đơn giản.

Kết quả tổng thể cho thấy hệ thống mới chỉ sử dụng khoảng 24% LUT, 15% FF, 24% BRAM và 20% DSP của FPGA Zynq UltraScale+ trên KV260. Điều này chứng minh thiết kế vẫn còn dư đáng kể tài nguyên phần cứng để mở rộng trong tương lai như tăng kích thước Systolic Array lên 32×32 , tăng số lượng DMA channel, bổ sung nhiều tầng buffering hoặc triển khai thêm các khối Attention và Feed-Forward Network của Transformer. Ngoài ra, việc mức sử dụng DSP chỉ ở khoảng 20% cho thấy nền tảng KV260 hoàn toàn đủ khả năng triển khai các bộ tăng tốc AI có quy mô lớn hơn trong tương lai.

3.3.2. Phân tích công suất tiêu thụ.

Sau quá trình Implementation, công cụ Vivado Power Analysis được sử dụng để ước lượng mức tiêu thụ công suất của toàn hệ thống dựa trên netlist sau Place & Route kết hợp với thông tin hoạt động của tín hiệu. Kết quả phân tích được trình bày trong Hình 20.



Hình 20. Kết quả phân tích hệ thống

Kết quả cho thấy tổng công suất tiêu thụ trên chip đạt khoảng $P_{total} = 2.995W$. Trong đó, công suất động $P_{dynamic} = 2.692W$ chiếm khoảng 90% tổng công suất hệ thống, còn công suất tĩnh $P_{static} = 0.302W$ chiếm khoảng 10%. Điều này phù hợp với đặc điểm của các hệ thống FPGA tăng tốc AI, nơi phần lớn năng lượng tiêu thụ đến từ hoạt động chuyển mạch của logic số và các khối DSP trong quá trình tính toán.

Phân tích chi tiết cho thấy phần PS tiêu thụ 2.419W tương đương khoảng 90% công suất động toàn hệ thống. Nguyên nhân chủ yếu đến từ bộ xử lý ARM Cortex-A53, hệ thống DDR controller, AXI interconnect, và các ngoại vi bên trong Zynq UltraScale+ MPSoC.

Trong khi đó, vùng PL chứa bộ tăng tốc GEMM chỉ tiêu thụ một phần nhỏ công suất mặc dù đang thực hiện đồng thời 256 phép nhân cộng INT8 trên mỗi chu kỳ clock.

Kết quả này cho thấy kiến trúc phần cứng đạt hiệu quả năng lượng cao nhờ tận dụng các khối DSP48E2 cứng thay vì xây dựng bộ nhân bằng LUT thông thường. Đồng thời, kiến trúc Systolic Array với luồng dữ liệu Output Stationary giúp giữ các giá trị cộng dồn trực tiếp bên trong PE, hạn chế đáng kể số lần truy cập bộ nhớ ngoài – vốn là thành phần tiêu thụ năng lượng lớn nhất trong các hệ thống AI accelerator. Ngoài ra,

việc sử dụng BRAM Ping-Pong, giao thức AXI4-Stream và pipeline dữ liệu liên tục giúp giảm trạng thái stall và hạn chế switching activity không cần thiết trong hệ thống. Công suất tiêu thụ của BRAM và DSP đều duy trì ở mức thấp cho thấy luồng dữ liệu bên trong bộ tăng tốc đã được tổ chức hiệu quả và phù hợp với đặc trưng xử lý Dense Matrix Multiplication của mô hình Transformer.

Bên cạnh đó, báo cáo nhiệt của Vivado cho thấy nhiệt độ junction của FPGA chỉ đạt 31.9°C với thermal margin = 53.1°C cho thấy hệ thống vẫn còn khoảng an toàn nhiệt lớn khi hoạt động ở tần số 100MHz.

Từ các kết quả trên có thể nhận thấy bộ tăng tốc GEMM không chỉ đáp ứng tốt về timing và tài nguyên phần cứng mà còn đạt hiệu quả năng lượng cao trên nền tảng AMD Kria KV260, phù hợp cho các ứng dụng Edge AI yêu cầu mức tiêu thụ công suất thấp.

Chương IV. THỰC NGHIỆM HỆ THỐNG TRÊN FPGA.

1. Trích xuất ma trận đầu vào và đánh giá dữ liệu lượng tử hóa.

Để phục vụ cho việc tối ưu hóa phần cứng, các ma trận đầu vào (bao gồm ma trận Kích hoạt - Activation A và ma trận Trọng số - Weight B) được lượng tử hóa đối xứng sang cấu trúc INT8 dựa trên phương pháp AbsMax. Tập dữ liệu trích xuất để kiểm thử bao gồm: cặp ma trận kích thước 512 x 512 từ lớp q_proj, cùng ma trận kích thước 5120 x 1280 và 1280 x 5120 từ lớp fc1.

```
=====
                          BÁO CÁO KÍCH THƯỚC DỮ LIỆU - LỚP Q_PROJ (ATTENTION)
=====
1. DỮ LIỆU THỎ TRÍCH XUẤT TỪ PYTORCH HOOK:
   - Activation gốc (act_raw) : [1500, 1280] (Thay đổi theo độ dài âm thanh)
-----
2. DỮ LIỆU ĐÃ ĐỊNH HÌNH CHO PHẦN CỨNG TĨNH (FP32):
   - Ma trận A (act_fp32)      : [512, 512]          | Mong đợi: [512, 512]
   - Ma trận B (weight_fp32)   : [512, 512]          | Mong đợi: [512, 512]
   - Ma trận C (Ground Truth) : [512, 512]          | Mong đợi: [512, 512]
-----
3. DỮ LIỆU SAU LƯỢNG TỬ HÓA - SẴN SÀNG XUẤT FILE BIN (INT8):
   - Ma trận A (act_int8)      : [512, 512]          | Kiểu: torch.int8
   - Ma trận B (weight_int8)   : [512, 512]          | Kiểu: torch.int8
=====

=====
                          BÁO CÁO KÍCH THƯỚC DỮ LIỆU - LỚP FC1 (UPSAMPLING)
=====
1. DỮ LIỆU THỎ TRÍCH XUẤT TỪ PYTORCH HOOK:
   - Activation gốc (act_raw) : [1500, 1280] (Thay đổi theo độ dài âm thanh)
-----
2. DỮ LIỆU ĐÃ ĐỊNH HÌNH CHO PHẦN CỨNG TĨNH (FP32):
   - Ma trận A (act_fp32)      : [5120, 1280]       | Mong đợi: [5120, 1280]
   - Ma trận B (weight_fp32)   : [1280, 5120]       | Mong đợi: [1280, 5120]
   - Ma trận C (Ground Truth) : [5120, 5120]       | Mong đợi: [5120, 5120]
-----
3. DỮ LIỆU SAU LƯỢNG TỬ HÓA - SẴN SÀNG XUẤT FILE BIN (INT8):
   - Ma trận A (act_int8)      : [5120, 1280]       | Kiểu: torch.int8
   - Ma trận B (weight_int8)   : [1280, 5120]       | Kiểu: torch.int8
=====

✅ TẤT CẢ MA TRẬN TỪ 2 LỚP ĐỀU ĐÃ VÀO ĐÚNG KÍCH THƯỚC CHUẨN THIẾT KẾ PHẦN CỨNG!
```

Hình 21. Kiểm tra kích thước ma trận đã trích xuất

Chất lượng của quá trình lượng tử hóa từ định dạng FP32 sang INT8 được đánh giá thông qua các chỉ số đo lường chuẩn công nghiệp. Kết quả đối chiếu được thể hiện chi tiết tại Hình 22.

Các Chỉ Số Đo Lường	Q_PROJ Act	Q_PROJ Wgt	FC1 Act	FC1 Wgt	Chuẩn Lý Thuyết
Độ tương đồng Cosine (Hướng)	0.999638	0.997565	1.000570	0.999047	> 0.990000
Tỷ số Tín hiệu/Nhiều (SQNR)	31.37 dB	23.10 dB	34.39 dB	24.74 dB	> 15.00 dB
Sai số tuyệt đối TB (MAE)	0.1614 LSB	0.2406 LSB	0.0684 LSB	0.2449 LSB	< 0.5000 LSB
Sai số bình phương gốc (RMSE)	0.2299 LSB	0.2812 LSB	0.1508 LSB	0.2854 LSB	Thấp (<0.6)
Mật độ phần tử bằng 0 (Sparsity)	43.24%	31.30%	73.83%	20.19%	Tự do
Dải giá trị thực tế sau Mapping	[-127, 59]	[-105, 127]	[-74, 127]	[-127, 108]	Trong [-128, 127]

Hình 22. Kết quả đối chiếu chất lượng lượng tử hóa đầu vào

Sai số toán học: Sai số tuyệt đối trung bình (MAE) của cả hai ma trận đều < 0.5 LSB, nằm hoàn hảo trong lưới số nguyên.

Độ tương đồng Cosine: Đạt mức cực cao (> 0.99), bảo toàn trọn vẹn hướng của các không gian vector nhúng.

Tỷ số SQNR: Vượt mốc 15 dB, khẳng định thông tin cốt lõi không bị méo.

Đánh giá trên phương diện lý thuyết, quá trình lượng tử hóa của các cặp ma trận đầu vào đã đáp ứng tốt các tiêu chuẩn kỹ thuật khắt khe. Dữ liệu sau lượng tử hóa duy trì được tính toàn vẹn cao, giúp giảm thiểu tối đa nguy cơ suy giảm độ chính xác (accuracy drop) khi thực thi mô hình Qwen2-Audio. Dữ liệu này đã hoàn toàn sẵn sàng để được nạp và triển khai trực tiếp trên các hệ thống phần cứng mạch RTL / FPGA.

2. Tích hợp phần mềm điều khiển.

Sau khi hoàn tất quá trình tổng hợp và thiết kế phần cứng trên FPGA, hệ thống phần mềm điều khiển được xây dựng trên môi trường hệ điều hành Linux nhúng của lõi xử lý ARM Cortex-A53. Phần mềm được phát triển bằng ngôn ngữ C, đóng vai trò là "bộ não" trung tâm điều phối toàn bộ luồng dữ liệu và tham số cấu hình cho bộ gia tốc GEMM thông qua hai giao thức chính: AXI DMA để truyền tải dữ liệu và AXI GPIO để cấu hình tham số tĩnh.

Chương trình điều khiển được thiết kế bài bản với các thành phần và chức năng cốt lõi sau:

2.1. Ánh xạ bộ nhớ vật lý không gian người dùng (Memory Mapping)

Hệ thống sử dụng cơ chế mmap thông qua tệp thiết bị hệ thống /dev/mem để ánh xạ trực tiếp các địa chỉ vật lý của phần cứng lên không gian bộ nhớ ảo của Linux. Quá trình này cấp quyền cho phần mềm C truy cập trực tiếp vào các thanh ghi điều khiển của 2 lõi DMA, 2 lõi GPIO và các phân vùng bộ nhớ DDR được cấp phát riêng biệt để chứa ma trận nguồn/đích.

2.2. Khởi tạo dữ liệu và cấu hình tham số động

Phần mềm đọc trực tiếp dữ liệu trọng số và kích hoạt lượng tử hóa từ các tệp nhị phân .bin đã được tiền xử lý từ mô hình Qwen2. Sau đó, thông qua AXI GPIO, phần mềm nạp các tham số vận hành vào bộ gia tốc bao gồm: chiều sâu khối tính toán ($K_DIM = 32$), tổng số vòng lặp cộng dồn ($NUM_K_TILES = 16$), hệ số dịch bit làm tròn ($SCALE_SHIFT = 10$) và tham số bù trừ ZERO_POINT.

2.3. Đóng gói dữ liệu theo kiến trúc Systolic Array.

Để đáp ứng luồng dữ liệu tối ưu của SA, phần mềm thực hiện chia nhỏ ma trận đầu vào thành các khối cơ sở 16×16 . Dữ liệu trước khi được DMA đẩy đi sẽ trải qua bước tái cấu trúc:

Khối ma trận A (Activation): Được trích xuất và lưu trữ tạm vào bộ đệm theo chuẩn Column-major. Điều này khớp hoàn hảo với luồng dữ liệu trôi theo chiều ngang của mảng phần cứng.

Khối ma trận B (Weight): Được trích xuất trực tiếp bằng lệnh memcopy theo chuẩn Row-major, tương thích với luồng dữ liệu trôi theo chiều dọc từ trên xuống của mảng.

2.4. Điều khiển truyền nhận DMA và cơ chế Polling an toàn

Quá trình truyền nhận dữ liệu được kích hoạt theo một quy trình nghiêm ngặt: Kênh nhận dữ liệu (S2MM) luôn được cấu hình địa chỉ và độ dài trước để đưa mảng phần cứng vào trạng thái sẵn sàng đón kết quả, sau đó kênh truyền (MM2S) mới bắt đầu đẩy ma trận A và B đi.

Để đảm bảo đồng bộ hóa giữa CPU và bộ tăng tốc, hệ thống sử dụng cơ chế Polling liên tục kiểm tra thanh ghi trạng thái. Điểm sáng của hệ thống là việc tích hợp cơ chế chống treo POLL_WAIT, giúp bảo vệ hệ điều hành Linux khỏi tình trạng đóng băng nếu có lỗi vật lý xảy ra trên bus dữ liệu.

2.5. Mô hình đối chiếu phần mềm.

Song song với việc gọi phần cứng, chương trình tích hợp một hàm nhân ma trận thuần túy chạy bằng CPU. Điểm đặc biệt của mô hình này là nó mô phỏng chính xác tuyệt đối logic đường ống phần cứng ở mức bit: thực hiện tích lũy số nguyên 32-bit (int32_t), cộng tín hiệu bias làm tròn, dịch bit và bão hòa dữ liệu về lại khoảng INT8 $[-128, 127]$. Kết quả của phần mềm được dùng làm cơ sở để đối chiếu từng phần tử với kết quả trả về từ thiết kế, đảm bảo tính đúng đắn chức năng tuyệt đối của hệ thống.

2.6. Đánh giá hiệu năng vi mô.

Thay vì chỉ đo đạc tổng thời gian, chương trình phần mềm chia nhỏ quá trình thực thi thành 4 giai đoạn micro-benchmarks để xác định chính xác nút thắt cổ chai:

T_{sw_prep} : Thời gian CPU thực hiện Tiling và Packing.

T_{dma_tx} : Thời gian DMA truyền dữ liệu và lỗi tính toán.

T_{dma_rx} : Thời gian chờ DMA ghi kết quả từ FIFO về DDR.

T_sw_post: Thời gian CPU gỡ gói dữ liệu và xếp lại thành ma trận kết quả hoàn chỉnh.

Kết quả thực nghiệm không chỉ trả về thông số thời gian chạy cứng thuần túy (Pure NPU Time) mà còn đo lường cả băng thông khả dụng thực tế của AXI-DMA, từ đó tính toán được tỷ lệ gia tốc của toàn bộ hệ thống so với vi xử lý ARM truyền thống. Cuối cùng, kết quả chạy bộ tăng được xuất thành tệp .bin để sẵn sàng cho các công đoạn kiểm định framework ở cấp độ cao hơn.

3. Vận hành bộ tăng tốc trên KV260.

Hệ thống được triển khai thực tế trên board AMD Kria KV260 chạy hệ điều hành Debian Linux nhúng. Việc giao tiếp với board được thực hiện từ máy tính thông qua kết nối SSH.

Bitstream của bộ tăng tốc được nạp vào vùng PL của FPGA bằng cơ chế FPGA Manager của Linux. File .bit sau khi được chép vào thư mục /lib/firmware/ sẽ được kích hoạt thông qua giao diện /sys/class/fpga_manager/fpga0/firmware. Trạng thái hoạt động của FPGA được kiểm tra thông qua file state, hệ thống chỉ tiếp tục chạy khi trạng thái hiển thị là operating.

Sau khi phần cứng được cấu hình thành công, các chương trình điều khiển viết bằng ngôn ngữ C được biên dịch trực tiếp trên KV260 bằng trình biên dịch GCC. Hai chương trình kiểm thử bao gồm: chương trình nhân ma trận kích thước 512×512 phục vụ kiểm tra chức năng và timing cơ bản, chương trình nhân ma trận kích thước $5120 \times 1280 \times 5120$ đánh giá khả năng xử lý của hệ thống với workload tương tự các phép toán trong mô hình AI.

Trong quá trình thực thi, phần mềm thực hiện:

1. Sinh dữ liệu INT8 ngẫu nhiên.
2. Thực hiện phép nhân ma trận trên CPU để làm baseline so sánh.
3. Cấu hình tham số cho bộ tăng tốc thông qua AXI GPIO.
4. Điều khiển AXI DMA truyền dữ liệu giữa DDR và PL.
5. Thu nhận kết quả tính toán từ bộ tăng tốc.
6. Đối chiếu kết quả giữa CPU và phần cứng.
7. Đo thời gian xử lý của CPU, phần cứng và toàn hệ thống.
8. Đo Transfer Time, Compute Time của thiết kế.

3.1. So sánh thời gian xử lý hệ thống trên testbench và trên KV260.

Thực hiện xử lý nhân ma trận $16 \times 512 \times 16$ để so sánh timing với testbench. Kết quả được hiển thị trên Hình 23 và Hình 24.



Hình 23. Thời gian xử lý nhân ma trận $16 \times 512 \times 16$ bằng testbench

```

debian@KV260-01:~/pduy/test_new_ver$
debian@KV260-01:~/pduy/test_new_ver$ gcc 16x512.c -o r16
debian@KV260-01:~/pduy/test_new_ver$ sudo ./r16

=====
>> Pure NPU Compute Time : 21710 ns (21.710 us)
=====
    
```

Hình 24. Thời gian phần cứng xử lý $16 \times 512 \times 16$ trên KV260

Thời gian hệ thống trên testbench là $14.28\mu s$ với chu kì $T = 10ns$. Tuy nhiên, thời gian xử lý trên phần cứng lại đạt $21.71\mu s$. Lí do chính là vì độ trễ vật lí, cũng như độ trễ của các bus được sử dụng trong thiết kế dẫn đến sai số $7.43\mu s$ này

3.2. Kiểm tra kết quả các ma trận lớn trên KV260.

Kết quả thực nghiệm trên Hình 25 và Hình 26 cho thấy hệ thống hoạt động ổn định trên phần cứng thực tế. Bộ tăng tốc GEMM thực hiện chính xác quá trình truyền dữ liệu, tính toán MAC, tái chuẩn hóa $INT32 \rightarrow INT8$ và ghi kết quả trở lại DDR RAM.

```
[1/4] Loading INT8 data from binary files... Done. Loaded 524288 bytes successfully.
[2/4] Running CPU Baseline Matrix Multiplication...
[CPU Baseline Progress] [=====] 100%
CPU Baseline Done in 4730.597 ms.

[3/4] Running NPU: 512x512 matmul | K_DIM=32 | TILES=16 | SHIFT=10

=====
>> PERFORMANCE & ACCURACY COMPARISON REPORT <<
=====
Matrix Size      : 512 x 512 x 512 (INT8)
1. CPU Execution Time      : 4730.597 ms
2. Pure NPU Hardware Time  : 47.381 ms (DMA TX + RX Wait)
3. Total NPU System Time   : 242.882 ms (Includes Linux overhead)
-----
>> SPEEDUP (Pure Hardware) : 99.84x FASTER than CPU
>> SPEEDUP (Total System)  : 19.48x FASTER than CPU
-----
>> SYSTEM LEVEL TIMING PROFILING REPORT <<
=====
1. Software Prep Time (T_sw_prep) : 190.622 ms ( 79.4%)
2. DMA Transmit + Core (T_dma_tx) : 45.733 ms ( 19.0%)
3. DMA Receive Time (T_dma_rx)    : 1.648 ms ( 0.7%)
4. Software Post Time (T_sw_post) : 2.154 ms ( 0.9%)
-----
>> Hardware Compute Efficiency : 19.73%
>> Effective DMA Bandwidth     : 342.96 MB/s
-----
>> ACCURACY CHECK              : [PASSED] NPU outputs perfectly match CPU!
=====

[4/4] Exporting hardware result to output_C512x512_NPU.bin... Done.
```

Hình 25. Kết quả xử lý 512x512 trên KV260

```
[1/4] Loading INT8 data from binary files... Done. Loaded 13107200 bytes successfully.
[2/4] Running CPU Baseline Matrix Multiplication (33.5 Billion MACs)...
[CPU Baseline Progress] [=====] 100%
CPU Baseline Done in 3634156.263 ms.

[3/4] Running NPU: 5120x5120 matmul | K_DIM=32 | TILES=40 | SHIFT=10

=====
>> PERFORMANCE & ACCURACY COMPARISON REPORT <<
=====
Matrix Size      : 5120 x 1280 x 5120 (INT8)
1. CPU Execution Time      : 3634156.263 ms
2. Pure NPU Hardware Time  : 11596.750 ms (DMA TX + RX Wait)
3. Total NPU System Time   : 84254.965 ms (Includes Linux overhead)
-----
>> SPEEDUP (Pure Hardware) : 313.38x FASTER than CPU
>> SPEEDUP (Total System)  : 43.13x FASTER than CPU
-----
>> SYSTEM LEVEL TIMING PROFILING REPORT <<
=====
1. Software Prep Time (T_sw_prep) : 71767.394 ms ( 85.8%)
2. DMA Transmit + Core (T_dma_tx) : 11431.208 ms ( 13.7%)
3. DMA Receive Time (T_dma_rx)    : 165.542 ms ( 0.2%)
4. Software Post Time (T_sw_post) : 245.649 ms ( 0.3%)
-----
>> Hardware Compute Efficiency : 13.87%
>> Effective DMA Bandwidth     : 347.08 MB/s
-----
>> ACCURACY CHECK              : [PASSED] NPU outputs perfectly match CPU!
=====

[4/4] Exporting hardware result to output_C5120x5120_NPU.bin... Done.
```

Hình 26. Kết quả xử lý 5120x1280x5120 trên KV260

Chương V. ĐÁNH GIÁ HỆ THỐNG.

1. Đánh giá hiệu năng xử lý giữa PS và bộ tăng tốc PL trên KV260

Để đánh giá hiệu năng của hệ thống, tiến hành so sánh giữa hai phương pháp xử lý:

Thực thi hoàn toàn trên bộ xử lý ARM Cortex-A53 (PS).

Thực thi bằng bộ gia tốc phần cứng trên FPGA (PL) kết hợp AXI DMA.

Việc đánh giá được thực hiện trực tiếp trên board KV260 với bài toán ma trận INT8 kích thước $512 \times 512 \times 512$ và $5120 \times 1280 \times 5120$. Kết quả thực nghiệm được thể hiện trong Hình 25 và Hình 26.

Từ Hình 25, đối với bài toán $512 \times 512 \times 512$:

Hệ thống vượt qua bài kiểm tra độ chính xác tuyệt đối. Kết quả phân cứng khớp 100% với mô phỏng phần mềm CPU. Điều này khẳng định logic phân cứng và luồng truyền DMA hoàn toàn không có lỗi thiết kế.

Thời gian chạy thuần túy trên phần cứng cực kỳ ấn tượng: bộ tăng tốc giải quyết bài toán chỉ trong 47.38 ms, nhanh gấp 99.84 lần so với việc chạy thuần trên CPU ARM (4730.6 ms).

Dù tốc độ tổng thể toàn hệ thống vẫn nhanh gấp 19.48 lần so với CPU, nhưng nút thắt lớn nhất đang nằm ở bộ vi xử lý ARM. Khâu chuẩn bị, tái cấu trúc và phân khối ma trận chiếm tới 79.4% tổng thời gian. Do CPU chuẩn bị dữ liệu quá lâu, thiết kế bị rơi vào trạng thái chờ dữ liệu, khiến hiệu suất tính toán phần cứng chỉ đạt mức 19.73%.

Kênh truyền AXI DMA đạt băng thông thực dụng 342.96 MB/s, mức khá tốt đối với SoC nhúng, đủ sức đẩy luồng dữ liệu liên tục cho mạch PL.

Từ Hình 26, đối với bài toán $5120 \times 1280 \times 5120$:

Mặc dù khối lượng tính toán khổng lồ lên tới 33.5 tỷ phép nhân cộng, hệ thống tiếp tục vượt qua bài kiểm tra độ chính xác tuyệt đối, chứng minh sự bền bỉ và độ tin cậy của thiết kế kiến trúc phần cứng khi hoạt động ở cường độ cao.

Ưu thế của bộ tăng tốc thể hiện sự vượt trội hoàn toàn khi ma trận phình to. Phần cứng chỉ mất khoảng 11.6 giây (11596.75 ms) để hoàn thành, trong khi CPU ARM phải mất tới hơn 1 giờ (3634156.26 ms). Tốc độ chạy cứng thuần túy nhanh gấp 313.38 lần so với CPU, cho thấy khả năng mở rộng xuất sắc của kiến trúc Systolic Array.

Tốc độ tổng thể toàn hệ thống (bao gồm cả overhead của Linux) vẫn nhanh gấp 43.13 lần so với CPU. Tuy nhiên, yếu điểm của PS càng bộc lộ rõ: khâu chuẩn bị phần mềm chiếm tới 85.8% tổng thời gian, kéo hiệu suất tính toán thực tế của phần cứng giảm xuống chỉ còn 13.87%.

Bảng thông AXI DMA thực dụng duy trì cực kỳ ổn định ở mức 347.08 MB/s, chứng tỏ khả năng truyền tải dữ liệu của hệ thống bus rất bền vững và không bị nghẽn suy giảm ngay cả khi phải luân chuyển hơn 13 MB dữ liệu đầu vào.

Thông qua hai kết quả thực nghiệm trên có thể thấy rằng việc chuyển khối lượng tính toán GEMM từ PS sang PL giúp cải thiện hiệu năng rất lớn, đặc biệt với các bài toán ma trận kích thước lớn có số lượng phép MAC cao. Đồng thời, hệ thống vẫn đảm bảo độ chính xác trong quá trình tính toán và tái chuẩn hóa dữ liệu INT32 về INT8 trên phần cứng.

2. Đánh giá tính đúng đắn và hiệu năng với Python

Để kiểm tra và đánh giá toàn diện độ chính xác của bộ tăng tốc GEMM trên kit KV260, hệ thống được thẩm định qua hai giai đoạn độc lập: Xác thực chéo logic phần cứng với mô hình chuẩn Golden Model và Đo lường chất lượng nhiễu lượng tử theo tiêu chuẩn công nghiệp dành cho mô hình ngôn ngữ lớn.

2.1. Xác thực chéo logic phần cứng và độ chính xác Bit-True

Quá trình kiểm định sử dụng một mô hình Golden Model được xây dựng trên framework PyTorch để mô phỏng chính xác từng bước xử lý trong thiết kế. Mô hình này tái hiện trọn vẹn chuỗi pipeline xử lý số nguyên bao gồm: phép nhân tích lũy trên thanh ghi INT32, khâu cộng sai số làm tròn, toán tử dịch bit phải, và cuối cùng là mạch bão hòa dữ liệu để đưa ma trận kết quả về định dạng INT8.

Kết quả đối chiếu trực tiếp giữa file nhị phân đầu ra thực tế từ chip KV260 và kết quả mô phỏng của Golden Model được thể hiện ở Hình 27.

BẢNG VẼ XÁC THỰC PHẦN CỨNG: BIẾN DỊCH TOÁN HỌC PYTORCH VS ĐẦU RA MẠCH RTL NPU				
Phân Lớp Mục Tiêu	Kích Thước Ma Trận	Số Điểm Lỗi Dữ Liệu	Sai Số Max (LSB)	Trạng Thái RTL
Attention Q PROJ	512x512 x 512x512	0 / 262,144	0 LSB	✅ MATCH 100%
FFN Upsample FC1	5120x1280 x 1280x5120	0 / 26,214,400	0 LSB	✅ MATCH 100%

Hình 27. Kết quả kiểm tra Bit-True

Kết quả này minh chứng hệ thống đạt trạng thái Đồng bộ hoàn hảo ở mức Bit-True. Điều này khẳng định logic điều khiển, các khối tính toán RTL trên FPGA và cấu trúc pipeline dịch bit/làm tròn trong thiết kế phần cứng hoàn toàn chính xác tuyệt đối về mặt toán học số nguyên so với framework chuẩn.

2.2. Phân tích sai số và chất lượng lượng tử hóa cấu trúc LLM

Bên cạnh việc xác thực logic số nguyên, chất lượng của thuật toán lượng tử hóa và mạch giải lượng tử ngược cũng được đánh giá bằng cách so sánh đầu ra của bộ tăng tốc với ma trận kết quả gốc FP32. Quá trình kiểm định được thực hiện chi tiết trên hai lớp mạng đặc trưng của mô hình Qwen2 là lớp chiếu q_proj (kích thước khối 512) và lớp mở rộng mạng FFN $fc1$ (kích thước khối 5120). Các chỉ số sai số thu được cụ thể như Hình 28.

Các Chỉ Số Đo Lường ($C = A \times B$)	Output Q_PROJ (512)	Output $FC1$ (5120)	Chuẩn LLM Lý Thuyết
Độ tương đồng Cosine (Hướng)	0.993575	0.998120	> 0.990000
Tỷ số Tín hiệu/Nhiều (SQNR)	18.88 dB	22.69 dB	> 15.00 dB
Sai số tuyệt đối TB (MAE Requant)	0.036134 FP32	0.035945 FP32	Nhỏ nhất có thể
Sai số bình phương gốc (RMSE)	0.043156 FP32	0.097420 FP32	Thấp (< 1.0)
Dải giá trị INT8 thu được trên NPU	[-19, 19]	[-53, 110]	Trong [-128, 127]

Hình 28. Kết quả đánh giá giải lượng tử kết quả GEMM

Từ kết quả thực nghiệm, chỉ số độ tương đồng Cosine của cả hai ma trận đầu ra đều vượt mốc tiêu chuẩn công nghiệp 0.99 (cụ thể đạt 0.993575 đối với lớp q_proj và 0.998120 đối với lớp $fc1$). Kết quả này chứng minh thiết kế đã bảo toàn nguyên vẹn đặc trưng không gian và hướng vector của các tầng ẩn trong mô hình LLM. Đồng thời, tỷ số tín hiệu trên nhiễu lượng tử (SQNR) đạt lần lượt 18.88 dB và 22.69 dB, vượt xa mốc lý thuyết yêu cầu (> 15.00 dB). Điều này khẳng định nhiễu biên độ sinh ra do quá trình ép kiểu từ FP32 xuống INT8 hoàn toàn nằm trong giới hạn hội tụ cho phép của cấu trúc Per-Tensor.

Bên cạnh đó, các chỉ số sai số tuyệt đối trung bình (MAE) sau khi giải lượng tử duy trì ở mức cực kỳ thấp (khoảng 0.036 cho cả hai lớp). Dải giá trị INT8 thu được trên thiết kế thực tế nằm an toàn trong khoảng [-128, 127] (chỉ dao động ở mức [-19, 19] với q_proj và [-53, 110] với $fc1$), hoàn toàn không xảy ra hiện tượng tràn số bão hòa. Hệ thống phần cứng kiểm soát rất tốt lỗi nhiễu lượng tử, đảm bảo đầu ra của cả khối Attention và khối FFN bám sát không gian vector gốc mà không gây ra hiện tượng Accuracy Drop hay sụp đổ mô hình khi triển khai.

Thông qua bộ chỉ số thực nghiệm khắt khe, thiết kế bộ tăng tốc GEMM trên nền tảng FPGA không chỉ đảm bảo tính đúng đắn tuyệt đối trong khâu thực thi logic phần cứng, mà còn chứng minh khả năng tối ưu hóa toán học xuất sắc, đáp ứng đầy đủ các tiêu chuẩn kỹ thuật khắt khe của công nghiệp AI/LLM về độ chính xác tín hiệu.

Chương VI. KẾT LUẬN.

Đồ án đã trình bày thành công quy trình nghiên cứu, thiết kế, hiện thực hóa và đánh giá một bộ tăng tốc phần cứng chuyên dụng cho phép nhân ma trận số nguyên trên nền tảng SoC FPGA AMD Kria KV260. Dựa trên kiến trúc Systolic Array kích thước 16×16 , hệ thống đã tích hợp hiệu quả cơ chế phân khối theo chiều sâu nhằm giải quyết triệt để bài toán xử lý các ma trận có kích thước khổng lồ, vượt ra khỏi giới hạn dung lượng lưu trữ cục bộ của tài nguyên on-chip.

Về mặt thiết kế phần cứng, đề tài đã xây dựng hoàn chỉnh hệ sinh thái các khối chức năng lõi bao gồm: Processing Element, MAC Pipeline, mạng định tuyến lệch pha dữ liệu, bộ điều khiển luồng, bộ tuần tự hóa đầu ra và module tái chuẩn hóa dữ liệu. Toàn bộ kiến trúc được mô tả ở mức thanh ghi bằng ngôn ngữ Verilog, trải qua quy trình kiểm chứng chức năng khắt khe trước khi tổng hợp và triển khai vật lý trên phân vùng Programmable Logic của vi mạch, đi kèm các phân tích chi tiết về tiêu thụ tài nguyên, đáp ứng xung nhịp và công suất hoạt động.

Về đánh giá hiệu năng trực tiếp trên KV260, các quy mô ma trận từ trung bình đến lớn đã minh chứng cho ưu thế vượt trội hoàn toàn của phân vùng logic lập trình (PL) so với bộ vi xử lý trung tâm (PS). Bộ tăng tốc đạt tốc độ xử lý phần cứng thuần túy cực kỳ ấn tượng, bỏ xa năng lực tính toán của CPU ARM và thể hiện khả năng mở rộng kiến trúc xuất sắc khi khối lượng dữ liệu phình to.

Về mặt đánh giá độ chính xác, kết quả thực nghiệm với các bài toán GEMM thực tế cho thấy hệ thống hoạt động cực kỳ ổn định với bài toán nhân ma trận kích thước 512×512 và $5120 \times 1280 \times 5120$. Kết quả đầu ra thực tế từ chip KV260 hoàn toàn trùng khớp ở mức Bit-True khi đối chiếu với mô hình Golden Model được xây dựng trên framework PyTorch. Khi so sánh với ma trận gốc FP32, bộ tăng tốc kiểm soát nhiễu lượng tử cực tốt với các chỉ số đạt chuẩn yêu cầu, chứng minh thiết kế bảo toàn nguyên vẹn đặc trưng không gian vector, đáp ứng hoàn hảo các yêu cầu khắt khe về độ chính xác để triển khai các mô hình LLM cấu trúc Per-Tensor mà không gây sụp đổ mô hình.

Bên cạnh các kết quả đạt được, hệ thống vẫn còn nhiều hướng phát triển trong tương lai. Hướng phát triển quan trọng nhất là đưa toàn bộ quá trình tiling và điều phối dữ liệu hiện đang được thực hiện trên PS xuống phần cứng PL. Việc tích hợp cơ chế scheduler và tile controller trực tiếp trong FPGA sẽ giúp giảm đáng kể overhead từ phần mềm Linux, hạn chế thời gian đóng gói dữ liệu và điều khiển DMA từ CPU, từ đó nâng cao hiệu năng tổng thể của hệ thống. Trong tương lai, bộ tăng tốc cũng có thể được tích hợp trực tiếp với các framework AI như ONNX Runtime hoặc Vitis AI để xây dựng các hệ thống suy luận AI hoàn chỉnh trên FPGA.

PHỤ LỤC

Source code thiết kế: [GitHub](#)

Slide báo cáo: [Slide](#)

NGUỒN THAM KHẢO

- [1] Qwen Team, "Qwen2 Technical Report," Alibaba Group, 2024. [Online]. Available: arXiv:2405.14458.
- [2] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [3] Quang-Chung Tran, "Qwen2-Audio-Dialogue Fine-tuned Model," Hugging Face Repository, 2024. [Online]. Available: <https://huggingface.co/tranquangchung/qwen2-audio-dialogue>.
- [4] X. Zhao *et al.*, "Linear Symmetric Quantization of Neural Networks for Low-precision Integer Hardware," in *International Conference on Learning Representations (ICLR)*, 2020.
- [5] S. Williams *et al.*, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65-76, 2009.
- [6] N. P. Jouppi *et al.*, "In-Datacenter Performance Analysis of a Tensor Processing Unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1-12.
- [7] B. Xu, A. Banerjee, and S. Gupta, "Hardware Acceleration for Neural Networks: A Comprehensive Survey," Arizona State University, 2024.
- [8] J. Li *et al.*, "Hummingbird+: Advancing FPGA-based LLM Deployment from Research Prototype to Edge Product," in *Proceedings of the ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA)*, 2026.
- [9] R. Kastner, J. Matai, and S. Neuendorffer, *Parallel Programming for FPGAs*, UC San Diego, 2018.
- [10] Advanced Micro Devices (AMD), *Kria KV260 Vision AI Starter Kit User Guide (UG1089)*, 2023.
- [11] A. Fengwei *et al.*, "A Component-Centric Perspective on Hardware Accelerators for LLMs," *IEEE Access*, vol. 12, 2024.
- [12] G. E. Moon *et al.*, "Evaluating Spatial Accelerator Architectures with Tiled Matrix-Matrix Multiplication," *IEEE Transactions on Parallel and Distributed Systems*, 2021.
- [13] T. Raja, "Systolic Array Data Flows for Efficient Matrix Multiplication in Deep Neural Networks," *arXiv preprint arXiv:2410.22595*, 2024.
- [14] Advanced Micro Devices (AMD), *Block Memory Generator v8.4 Product Guide (PG058)*, 2021.
- [15] Advanced Micro Devices (AMD), *AXI DMA LogiCORE IP Product Guide (PG021)*, 2025.
- [16] Arm Limited, "AMBA® AXI Protocol Specification," ARM IHI 0022, Issue L, Aug. 27, 2025.

- [17] Real Digital, "Introduction to AXI4-Lite Interface," Tech. Rep., 2025.
- [18] ARM, *AMBA 4 AXI4-Stream Protocol Specification (Version 1.0)*, ARM Ltd., 2010.
- [19] D. Baptista, L. Sousa, and F. Morgado-Dias, "Raising the Abstraction Level of a Deep Learning Design on FPGAs," *IEEE Access*, vol. 8, pp. 205151-205161, 2020, doi: 10.1109/ACCESS.2020.3036975